

1 Pancake Sorting Problem

Definition 1.1 (Pancake numbers).

We are given a stack of n pancakes, each of different size. Our goal is to sort this stack from smallest to largest (largest being on the bottom of the stack). The only thing we are allowed to do is to insert the spatula in between two pancakes (or between the bottom pancake and the plate), and flip over all the pancakes that are on top of the spatula.



We are interested in the maximum number of flips (in terms of n) we would need to sort a stack of n pancakes, where the maximum is over all stacks with n pancakes. In other words, we are interested in

$$P_n = \max_S \min_A \text{ number of flips that method } A \text{ takes to sort stack } S.$$

Here, the maximum is over all pancake stacks of size n , and the minimum is over all methods/algorithms for sorting a given stack of pancakes. ■

Notation 1.2. We represent a stack of n pancakes with a permutation of $\{1, 2, \dots, n\}$. Here, the numbers correspond to how large the pancake is, so 1 represents the smallest pancake and n represents the largest pancake. For example, $(5\ 2\ 3\ 4\ 1)$ corresponds to a stack of 5 pancakes, where the largest pancake 5 is at the top of the stack, and the smallest pancake 1 is at the bottom.

Proposition 1.3 (Number of flips required for $(5\ 2\ 3\ 4\ 1)$). *Let X denote the minimum number of flips needed to sort the stack $(5\ 2\ 3\ 4\ 1)$. Then $X = 4$.*

Proof. To prove $X \leq 4$, we show how to sort $(5\ 2\ 3\ 4\ 1)$ in 4 flips:

$$(5\ 2\ 3\ 4\ 1) \rightarrow (1\ 4\ 3\ 2\ 5) \rightarrow (2\ 3\ 4\ 1\ 5) \rightarrow (4\ 3\ 2\ 1\ 5) \rightarrow (1\ 2\ 3\ 4\ 5).$$

We now prove $X \geq 4$. The proof is by contradiction, so assume that there is a way to sort $(5\ 2\ 3\ 4\ 1)$ in 3 or less flips.

Observation. Right before a pancake is placed at the bottom of the stack, it must be placed at the top of the stack.

Claim. The first flip must put 5 on the bottom of the stack.

Proof of Claim. Suppose the first flip does not put 5 on the bottom of the stack, so it puts it somewhere in the middle. Then we can show that $(5\ 2\ 3\ 4\ 1)$ cannot be sorted in 3 or less flips. We know that after 3 flips, 5 must be placed at the bottom of the

stack. The observation above implies that the second flip must send 5 to the top. So in the first two flips, 5 first gets sent from the top to somewhere in the middle, and then it gets flipped back up to the top. In other words, after 2 flips we end up with the original stack $(5\ 2\ 3\ 4\ 1)$. There is no way to sort $(5\ 2\ 3\ 4\ 1)$ with the remaining flip, which proves the claim.

So we know that the first flip must be $(5\ 2\ 3\ 4\ 1) \rightarrow (1\ 4\ 3\ 2\ 5)$. In the remaining two flips, 4 must be placed next to 5. It is clear that 5 should not be touched (i.e., we should not be flipping the whole stack). So we can ignore 5 and just consider the stack of 4 pancakes $(1\ 4\ 3\ 2)$. We need to put 4 at the bottom of this stack in 2 flips. Again, using the observation stated above, we know that 4 must be first placed at the top of the stack. So the 2 flips must be $(1\ 4\ 3\ 2) \rightarrow (4\ 1\ 3\ 2) \rightarrow (2\ 3\ 1\ 4)$. The resulting stack is not sorted, which is the desired contradiction. □

Theorem 1.4. *For $n \geq 4$, we have*

$$n \leq P_n \leq 2n - 3.$$

The proof of the theorem follows from the following two lemmas.

Lemma 1.5. *For $n \geq 2$, we have $P_n \leq 2n - 3$.*

Proof. Consider the following algorithm for sorting an arbitrary stack of n pancakes.

- If $n = 1$: do nothing.
- If $n = 2$: sort the pancakes in one flip if they are not already sorted.
- Else (if $n \geq 3$):
 - Bring the largest pancake to the bottom of the stack in 2 flips.
 - Recursively sort the remaining $n - 1$ pancakes.

Clearly,¹ the algorithm correctly sorts a given stack of pancakes. Let $T(n)$ be the number of flips that this algorithm uses to sort a stack of n pancakes. By the definition of P_n , $P_n \leq T(n)$. So we are done once we show $T(n) \leq 2n - 3$ for $n \geq 2$. The recursive relation that $T(n)$ satisfies is

$$\begin{aligned} T(1) &= 0, \\ T(2) &\leq 1, \\ T(n) &\leq 2 + T(n - 1) \quad \text{for } n \geq 3. \end{aligned}$$

This implies that $T(n) \leq 2n - 3$ for $n \geq 2$, which completes the proof.² □

¹You should be careful using the word “clearly”. In this case, it is justified.

²To be more complete, you can prove $T(n) \leq 2n - 3$ for $n \geq 2$ with a quick induction. This part is omitted.

Exercise 1.6. Show by induction that the recurrence relation in the above proof solves to $T(n) \leq 2n - 3$ for $n \geq 2$.

Lemma 1.7. For $n \geq 4$, we have $P_n \geq n$.

Proof. Given $i, j \in \{1, 2, \dots, n\}$ and a pancake stack, we say that (i, j) form a *bad pair* with respect to that stack if they are adjacent in the stack, and $|i - j| > 1$ (i.e., they are not supposed to be adjacent once the stack is sorted). Observe that if two pancakes are adjacent in a stack, they will remain adjacent if the spatula is never inserted in between them. This means that if (i, j) form a bad pair, then any sorting method that sorts the stack *must* insert the spatula in between i and j at some point. Note that we can also consider the bottom pancake and the plate as a bad pair too. If we never insert the spatula at the bottom of the stack, then the bottom pancake and the plate will remain adjacent. So we extend our definition of a bad pair to include the plate too.

Now we can conclude that a stack with b bad pairs needs at least b flips to be sorted. We finish the proof by showing that for $n \geq 4$, there is a stack of n pancakes containing n bad pairs. We do this by considering two cases: when n is even and when n is odd. When n is even, the following stack has n bad pairs:

$$(2 \ 4 \ 6 \ \dots \ n - 2 \ n \ 1 \ 3 \ 5 \ \dots \ n - 1).$$

When n is odd, the following stack has n bad pairs:

$$(1 \ 3 \ 5 \ \dots \ n - 2 \ n \ 2 \ 4 \ 6 \ \dots \ n - 1).$$

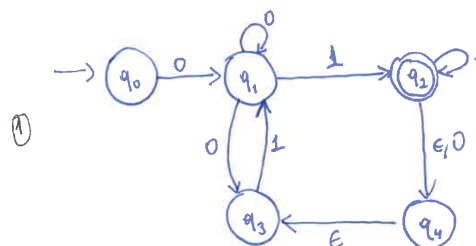
(Note that the assumption $n \geq 4$ is required so that the pancakes right in the middle of the stacks form a bad pair.) $\square$

Exercise 1.8. Suppose we are allowed to take any contiguous set of pancakes and flip them in place (they need not be on the top of the stack). Let Q_n be the maximum over stacks of size n of the minimum number of flips required to sort that stack, using this new flipping operation. Show that $n/2 \leq Q_n \leq n - 1$ for all $n \geq 2$.

15-252 Spring 2018 Non-Deterministic Finite Automata (NFAs)

Intro :

- non-determinism important concept in CS. Does it give extra power? P vs NP (NON-DET $\neq$ RANDOMIZATION)
- many closure properties of reg. languages can be proved very easily with NFAs. (Often math is all about coming up with the right definitions.)



001 accepts

④ 01011 accepts

00100 rejects

Comparison with DFAs:

- * not all ~~possible~~ possible transitions present.
- * multiple transitions with same label from a state.
- * ϵ transitions.

Rule for accepting/rejecting:

- ③
- Accept w if there is a "valid" path that leads to an accepting state.
 - Reject w if all paths lead to a rejecting state.

Example computation trace

	0	0	1	1	0
q_0	q_1	q_1	q_2	q_2	q_4
		q_3	q_1	q_2	
			q_4		
		q_3	q_1	q_1	
			q_4		
			q_3		
			q_4		
			q_3		
				q_3	
				q_3	

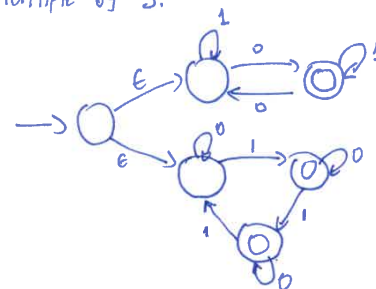
Def: An NFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where

- Q finite non-empty set (set of states)
- Σ " " (alphabet)
- $\delta: Q \times \Sigma \cup \{\epsilon\} \rightarrow \mathcal{P}(Q)$ (transition function)
- $q_0 \in Q$ (starting state)
- $F \subseteq Q$ (set of accepting states)

Exercise: Define formally the notion of an NFA accepting a string.

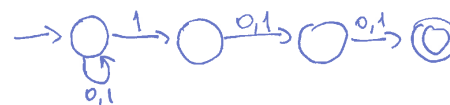
Drawing NFAs

All binary strings that have either odd # 0's or #1's is not a multiple of 3.



Do this LAST!

$L = \{w \in \{0,1\}^* : |w| \geq 3 \text{ and } w_{n-2} = 1 \text{ where } n = |w|\}$



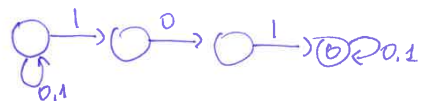
$L_k = \{w \in \{0,1\}^* : |w| \geq k \text{ and } w_{n-(k-1)} = 1 \text{ where } n = |w|\}$

Best DFA needs 2^k states.

$L =$ all binary string that end with 101.



$L =$ all binary strings that contain 101 as a substring.



Thm: $NFA \equiv DFA$ (any language recognized by a NFA is regular;

Proof Sketch: Threading idea works.

Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA. (Suppose N has no ϵ -transitions.) Construct DFA $M = (Q', \Sigma, \delta', q_0', F')$ as follows:

$$Q' = \mathcal{P}(Q)$$

$$\delta' : Q' \times \Sigma \rightarrow Q', \quad \forall S \in Q', a \in \Sigma, \delta'(S, a) = \bigcup_{s \in S} \delta(s, a)$$

$$q_0' = \{q_0\}$$

$$F' = \{S \in Q' : S \cap F \neq \emptyset\}$$

if N has ϵ -transitions, define, for $S \in Q'$, $E(S) = \{q \in Q : q \text{ can be reached from a state in } S \text{ using zero or more } \epsilon\text{-transitions}\}$

$$\delta''(S, a) = E(\delta'(S, a))$$

$$q_0'' = E(\{q_0\})$$

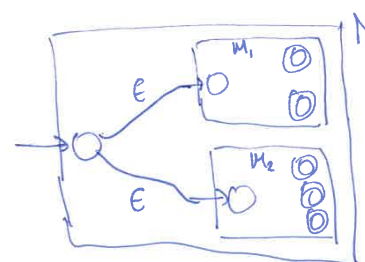
COMMENT: # STATES IN DFAs VS NFAs

Closure Properties of Regular Languages

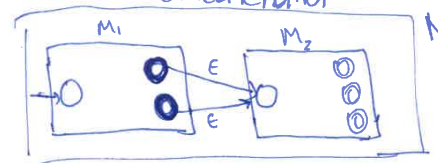
Closure under union:



build NFA

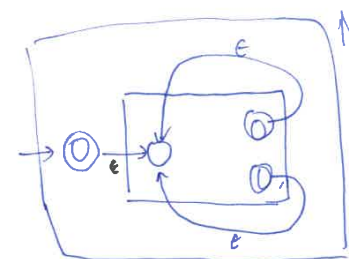
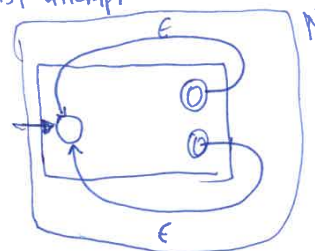


Closure under concatenation

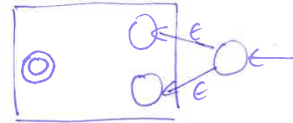
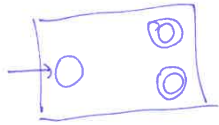


Closure under star

First attempt



Closure under reversal



reverse every transition.

Lecture Notes on The Lambda Calculus

15-814: Types and Programming Languages
Frank Pfenning

Lecture 1
Tuesday, September 4, 2018

1 Introduction

This course is about the principles of programming language design, many of which derive from the notion of *type*. Nevertheless, we will start by studying an exceedingly pure notion of computation based only on the notion of function, that is, Church's λ -calculus [CR36]. There are several reasons to do so.

- We will see a number of important concepts in their simplest possible form, which means we can discuss them in full detail. We will then reuse these notions frequently throughout the course without the same level of detail.
- The λ -calculus is of great historical and foundational significance. The independent and nearly simultaneous development of Turing Machines [Tur36] and the λ -Calculus [CR36] as universal computational mechanisms led to the *Church-Turing Thesis*, which states that the effectively computable (partial) functions are exactly those that can be implemented by Turing Machines or, equivalently, in the λ -calculus.
- The notion of function is the most basic abstraction present in nearly all programming languages. If we are to study programming languages, we therefore must strive to understand the notion of function.
- It's cool!

2 The λ -Calculus

In ordinary mathematical practice, functions are ubiquitous. For example, we might define

$$\begin{aligned} f(x) &= x + 5 \\ g(y) &= 2 * y + 7 \end{aligned}$$

Oddly, we never state what f or g actually are, we only state what happens when we apply them to arbitrary arguments such as x or y . The λ -calculus starts with the simple idea that we should have notation for the function itself, the so-called λ -abstraction.

$$\begin{aligned} f &= \lambda x. x + 5 \\ g &= \lambda y. 2 * y + 7 \end{aligned}$$

In general, $\lambda x. e$ for some arbitrary expression e stands for the function which, when applied to some e' becomes $[e'/x]e$, that is, the result of *substituting* or *plugging in* e' for occurrences of the variable x in e . For now, we will use this notion of substitution informally—in the next lecture we will define it formally.

We can already see that in a pure calculus of functions we will need at least three different kinds of expressions: λ -abstractions $\lambda x. e$ to form function, *application* $e_1 e_2$ to apply a function e_1 to an argument e_2 , and *variables* x, y, z , etc. We summarize this in the following form

$$\begin{array}{ll} \text{Variables} & x \\ \text{Expressions } e & ::= \lambda x. e \mid e_1 e_2 \mid x \end{array}$$

This is not the definition of the *concrete syntax* of a programming language, but a slightly more abstract form called *abstract syntax*. When we write down concrete expressions there are additional conventions and notations such as parentheses to avoid ambiguity.

1. Juxtaposition (which expresses application) is *left-associative* so that $x y z$ is read as $(x y) z$
2. $\lambda x.$ is a prefix whose scope extends as far as possible while remaining consistent with the parentheses that are present. For example, $\lambda x. (\lambda y. x y z) x$ is read as $\lambda x. ((\lambda y. (x y) z) x)$.

We say $\lambda x. e$ *binds* the variable x with scope e . Variables that occur in e but are not bound are called *free variables*, and we say that a variable x may occur free in an expression e . For example, y is free in $\lambda x. x y$ but

not x . Bound variables can be renamed consistently in a term So $\lambda x. x + 5 = \lambda y. y + 5 = \lambda \text{whatever}. \text{whatever} + 5$. Generally, we rename variables *silently* because we identify terms that differ only in the names of λ -bound variables. But, if we want to make the step explicit, we call it α -conversion.

$$\lambda x. e =_{\alpha} \lambda y. [y/x]e \quad \text{provided } y \text{ not free in } e$$

The proviso is necessary, for example, because $\lambda x. x y \neq \lambda y. y y$.

We capture the rule for function application with

$$(\lambda x. e_2) e_1 =_{\beta} [e_1/x]e_2$$

and call it β -conversion. Some care has to be taken for the substitution to be carried out correctly—we will return to this point later.

If we think beyond mere equality at *computation*, we see that β -conversion has a definitive direction: we apply is from left to right. We call this β -*reduction* and it is the engine of computation in the λ -calculus.

$$(\lambda x. e_2) e_1 \longrightarrow_{\beta} [e_1/x]e_2$$

3 Function Composition

One the most fundamental operation on functions in mathematics is to compose them. We might write

$$(f \circ g)(x) = f(g(x))$$

Having λ -notation we can first explicitly denote the result of composition (with some redundant parentheses)

$$f \circ g = \lambda x. f(g(x))$$

As a second step, we realize that $\circ$ itself is a function, taking two functions as arguments and returning another function. Ignoring the fact that it is usually written in infix notation, we define

$$\circ = \lambda f. \lambda g. \lambda x. f(g(x))$$

Now we can calculate, for example, the composition of the two functions we had at the beginning of the lecture. We note the steps where we apply

β -conversion.

$$\begin{aligned}
 & (\circ (\lambda x. x + 5)) (\lambda y. 2 * y + 7) \\
 = & ((\lambda f. \lambda g. \lambda x. f(g(x))) (\lambda x. x + 5)) (\lambda y. 2 * y + 7) \\
 =_{\beta} & (\lambda g. \lambda x. (\lambda x. x + 5)(g(x))) (\lambda y. 2 * y + 7) \\
 =_{\beta} & \lambda x. (\lambda x. x + 5) ((\lambda y. 2 * y + 7)(x)) \\
 =_{\beta} & \lambda x. (\lambda x. x + 5) (2 * x + 7) \\
 =_{\beta} & \lambda x. (2 * x + 7) + 5 \\
 = & \lambda x. 2 * x + 12
 \end{aligned}$$

While this appears to go beyond the pure λ -calculus, we will see in Section 7 that we can actually encode natural numbers, addition, and multiplication. We can see that $\circ$ as an operator is not *commutative* because, in general, $\circ f g \neq \circ g f$. You may test your understanding by calculating $(\circ (\lambda y. 2 * y + 7)) (\lambda x. x + 5)$ and observing that it is different.

4 Identity

The simplest function is the identity function

$$I = \lambda x. x$$

We would expect that in general, $\circ I f = f = \circ f I$. Let's calculate one of these:

$$\begin{aligned}
 & \circ I f \\
 = & ((\lambda f. \lambda g. \lambda x. f(g(x))) (\lambda x. x)) f \\
 =_{\beta} & (\lambda g. \lambda x. (\lambda x. x)(g(x))) f \\
 =_{\beta} & \lambda x. ((\lambda x. x)(f(x))) \\
 =_{\beta} & \lambda x. f(x)
 \end{aligned}$$

We see $\circ I f = \lambda x. f x$ but it does not appear to be equal to f . However, $\lambda x. f x$ and f would seem to be equal in the following sense: if we apply both sides to an arbitrary expression e we get $(\lambda x. f x) e = f e$ on the left and $f e$ on the right. In other words, the two functions appear to be *extensionally* equal. We capture this by adding another rule to the calculus call η .

$$e =_{\eta} \lambda x. e x \quad \text{provided } x \text{ not free in } e$$

The proviso is necessary—can you find a counterexample to the equality if it is violated?

5 Summary of λ -Calculus

λ -Expressions.

$$\begin{array}{ll}
 \text{Variables} & x \\
 \text{Expressions} & e ::= \lambda x. e \mid e_1 e_2 \mid x
 \end{array}$$

$\lambda x. e$ binds x with scope e , which is as large as possible while remaining consistent with the given parentheses. Juxtaposition $e_1 e_2$ is left-associative.

Equality.

$$\begin{array}{llll}
 \text{Substitution} & [e_1/x]e_2 & & (\text{capture-avoiding, see Lecture 2}) \\
 \alpha\text{-conversion} & \lambda x. e & =_{\alpha} & \lambda y. [y/x]e \quad \text{provided } y \text{ not free in } e \\
 \beta\text{-conversion} & (\lambda x. e_2) e_1 & =_{\beta} & [e_1/x]e_2 \\
 \eta\text{-conversion} & \lambda x. e x & =_{\eta} & e \quad \text{provided } x \text{ not free in } e
 \end{array}$$

We generally apply α -conversion silently, identifying terms that differ only in the names of the bound variables. When we write $e = e'$ we allow $\alpha\beta\eta$ -equality and the usual mathematical operations such as expanding a definition.

Reduction.

$$\beta\text{-reduction} \quad (\lambda x. e_2) e_1 \longrightarrow_{\beta} [e_1/x]e_2$$

6 Representing Booleans

Before we can claim the λ -calculus as a universal language for computation, we need to be able to represent *data*. The simplest nontrivial data type are the Booleans, a type with two elements: *true* and *false*. The general technique is to represent the values of a given type by *normal forms*, that is, expressions that cannot be reduced. Furthermore, they should be *closed*, that is, not contain any free variables. We need to be able to distinguish between two values, and in a closed expression that suggest introducing two bound variables. We then define rather arbitrarily one to be *true* and the other to be *false*

$$\begin{aligned}
 \text{true} &= \lambda x. \lambda y. x \\
 \text{false} &= \lambda x. \lambda y. y
 \end{aligned}$$

The next step will be to define *functions* on values of the type. Let's start with negation: we are trying to define a λ -expression *not* such that

$$\begin{aligned} \text{not true} &= \text{false} \\ \text{not false} &= \text{true} \end{aligned}$$

We start with the obvious:

$$\text{not} = \lambda b. \dots$$

Now there are two possibilities: we could either try to apply b to some arguments, or we could build some λ -abstractions. In lecture, we followed the first path—you may want try the second as an exercise.

$$\text{not} = \lambda b. b (\dots) (\dots)$$

We suggest two arguments to b , because b stands for a Boolean, and Booleans *true* and *false* both take two arguments. $\text{true} = \lambda x. \lambda y. x$ will pick out the first of these two arguments and discard the second, so since we specified $\text{not true} = \text{false}$, the first argument to b should be *false*!

$$\text{not} = \lambda b. b \text{ false } (\dots)$$

Since $\text{false} = \lambda x. \lambda y. y$ picks out the second argument and $\text{not false} = \text{true}$, the second argument to b should be *true*.

$$\text{not} = \lambda b. b \text{ false true}$$

Now it is a simple matter to calculate that the computation of *not* applied to *true* or *false* completes in three steps and obtain the correct result.

$$\begin{aligned} \text{not true} &\longrightarrow_{\beta}^3 \text{false} \\ \text{not false} &\longrightarrow_{\beta}^3 \text{true} \end{aligned}$$

We write $\longrightarrow_{\beta}^n$ for reduction in n steps, and $\longrightarrow_{\beta}^*$ for reduction in an arbitrary number of steps, including zero steps. In other words, $\longrightarrow_{\beta}^*$ is the reflexive and transitive closure of $\longrightarrow_{\beta}$.

As a next exercise we try conjunction. We want to define a λ -expression *and* such that

$$\begin{aligned} \text{and true true} &= \text{true} \\ \text{and true false} &= \text{false} \\ \text{and false true} &= \text{false} \\ \text{and false false} &= \text{false} \end{aligned}$$

Learning from the negation, we start by guessing

$$\text{and} = \lambda b. \lambda c. b (\dots) (\dots)$$

where we arbitrarily put b first. If b is *true*, this will return the first argument. Looking at the equations we see that this should always be equal to the second argument.

$$\text{and} = \lambda b. \lambda c. b c (\dots)$$

If b is *false* the result is always false, no matter what c is, so the second argument to b is just *false*.

$$\text{and} = \lambda b. \lambda c. b c \text{ false}$$

Again, it is now a simple matter to verify the desired equations and that, in fact, the right-hand side of these equations is obtained by reduction.

We know we can represent all functions on Booleans returning Booleans once we have negation and conjunction. But we can also represent the more general conditional *if* with the requirements

$$\begin{aligned} \text{if true } u \ w &= u \\ \text{if false } u \ w &= w \end{aligned}$$

Note here that the variable u and w stand for arbitrary λ -expressions and not just Booleans. From what we have seen before, the conditional is now easy to define:

$$\text{if} = \lambda b. \lambda u. \lambda w. b u w$$

Looking at the innermost abstraction, we have $\lambda w. (b u) w$ which is actually η -convertible to $b u$! Taking another step we arrive at

$$\begin{aligned} \text{if} &= \lambda b. \lambda u. \lambda w. b u w \\ &=_{\eta} \lambda b. \lambda u. b u \\ &=_{\eta} \lambda b. b \\ &= I \end{aligned}$$

In other words, the conditional is just the identity function!

7 Representing Natural Numbers

Finite types such as Booleans are not particularly interesting. When we think about the computational power of a calculus we generally consider

the *natural numbers* $0, 1, 2, \dots$. We would like a representation $\bar{n}$ such that they are all distinct. We obtain this by thinking of the natural numbers are generated from zero by repeated application of the successor function. Since we want our representations to be closed we start with two abstractions: one (z) that stands for zero, and one (s) that stands for the successor function.

$$\begin{aligned}\bar{0} &= \lambda z. \lambda s. z \\ \bar{1} &= \lambda z. \lambda s. s z \\ \bar{2} &= \lambda z. \lambda s. s (s z) \\ \bar{3} &= \lambda z. \lambda s. s (s (s z)) \\ \dots & \\ \bar{n} &= \lambda z. \lambda s. \underbrace{s(\dots(s z))}_{n \text{ times}}\end{aligned}$$

In other words, the representation $\bar{n}$ iterates its second argument n times over its first argument

$$\bar{n} x f = f^n(x)$$

where $f^n(x) = \underbrace{f(\dots(f(x)))}_{n \text{ times}}$

The first order of business now is to define a successor function that satisfies $\text{succ } \bar{n} = \bar{n} + 1$. As usual, there is more than one way to define it, here is one (throwing in the definition of *zero* for uniformity):

$$\begin{aligned}\text{zero} &= \bar{0} \\ \text{succ} &= \lambda n. \lambda z. \lambda s. \bar{n} + 1 = \lambda n. \lambda z. \lambda s. s (\bar{n} z s)\end{aligned}$$

We cannot carry out the correctness proof in closed form as we did for the Booleans since there would be infinitely many cases to consider. Instead we calculate generically (using mathematical notation and properties)

$$\begin{aligned}\text{succ } \bar{n} &= \lambda z. \lambda s. s (\bar{n} z s) \\ &= \lambda z. \lambda s. s (s^n(z)) \\ &= \lambda z. \lambda s. s^{n+1}(z) \\ &= \bar{n} + 1\end{aligned}$$

A more formal argument might use mathematical induction over n .

Using the iteration property we can now define other mathematical functions over the natural numbers. For example, addition of n and k iterates the successor function n times on k .

$$\text{plus} = \lambda n. \lambda k. n \text{ succ } k$$

You are invited to verify the correctness of this definition by calculation. Similarly:

$$\begin{aligned}\text{times} &= \lambda n. \lambda k. n \text{ (plus } k) \text{ zero} \\ \text{exp} &= \lambda b. \lambda e. e \text{ (times } b) \text{ (succ zero)}\end{aligned}$$

Everything appears to be going swimmingly until we hit the predecessor function defined by

$$\begin{aligned}\text{pred } \bar{0} &= \bar{0} \\ \text{pred } \bar{n} + 1 &= \bar{n}\end{aligned}$$

You may try for a while to see if you can define the predecessor function, but it is very difficult. The problem seems to be that it is easy to define functions f using the schema of *iteration*

$$\begin{aligned}f 0 &= c \\ f (n + 1) &= g (f n)\end{aligned}$$

(namely: $f = \lambda n. n c g$), but not the so-called schema of *primitive recursion*

$$\begin{aligned}f 0 &= c \\ f (n + 1) &= g n (f n)\end{aligned}$$

because it is difficult to get access to n .

More about this and other properties and examples of the λ -calculus in Lecture 2.

References

- [CR36] Alonzo Church and J.B. Rosser. Some properties of conversion. *Transactions of the American Mathematical Society*, 39(3):472–482, May 1936.
- [Tur36] Alan Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42:230–265, 1936. Published 1937.

CS 252, Lecture 4: Kolmogorov Complexity

Nicolas Resch

February 6, 2020

1 Introduction & Motivation

The goal of *Kolmogorov* (or *Kolmogorov-Chaitin* or *descriptive*) complexity is to develop a measure of the complexity or “randomness” of an object. We would furthermore like this measure to be intrinsic to the object in question. For concreteness, the objects we consider will be strings over a fixed alphabet, but this is no real loss of generality. Thus, just as computability/complexity theory aims to quantify the complexity of languages (i.e., sets of strings), Kolmogorov complexity aims to quantify the complexity of individual strings.

Consider the following strings:

$A = 4444444444$

$B = 2718281828$

$C = 1756475382$

Which string is the most random? From the point-of-view of probability, if every 10 digit string has the same likelihood of being sampled, the strings are all “equally” random: each is sampled with probability 10^{-10} .

However, intuitively, string A appears to be very simple, in the sense that it has a short description: it is simply the digit 4 repeated 10 times. Strings B and C , on the other hand, appear to be much more complicated; one might even be tempted to say that they appear “random”. However, B actually also has a fairly simple description: it is the first 10 digits in the decimal expansion of Euler’s number! C , on the other hand, is indeed the result of randomly sampling a 10 digit number.

This discussion suggests that the Kolmogorov complexity of a string should be somehow linked to the length of its shortest description. Even with this intuition in mind, coming up with a meaningful definition is quite subtle. For example, consider the so-called Richard-Berry paradox: we describe a natural number as “the least natural number that cannot be described in less than twenty words”. If this number exists, then we have just described it with thirteen words, a contradiction to its definition. On the other hand, if such a number does not exist, that means that every number can be described in fewer than twenty words, which is equally absurd!

Thus, we need to be very careful about what we consider to be a valid description. Moreover, we would like the definition to be “fair” in the sense that it doesn’t artificially favor certain objects with short

descriptions. With these motivations and caveats in mind, let us begin to develop the theory of Kolmogorov complexity.

2 Formal Theory

2.1 Description Methods

Abstractly, a *description method* is just a partial function $f : \Sigma^* \rightarrow \Sigma^*$ (with $\Sigma = \{0, 1\}$, say) mapping descriptions to objects. The complexity of string x under description method f is

$$C_f(x) := \min\{|p| : f(p) = x\}.$$

While this is a good start, we promised a measure of a string’s complexity that depends only on x . However, $C_f(x)$ clearly depends on f ! For this reason, $C_f(x)$ could arbitrarily favor certain strings with short descriptions. As an example, define $f(0) = 011101110111000110011$ and $f(1x) = x$ for all other strings x . Under this description method, $C_f(011101110111000110011) = 1$ whereas $C_f(x) = 1 + |x|$ for all other strings x . To remedy this, we will need to settle upon a *universal* description method.

2.2 Universal Description Method

Our goal is to pick a description method that is the “best” in the sense that it gives the minimum description length to all strings. The following definition formalizes this.

Definition 1 (Minorization). A partial function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ *minorizes* a partial function $g : \{0, 1\}^* \rightarrow \{0, 1\}^*$ if there exists a constant c such that for all strings x ,

$$C_f(x) \leq C_g(x) + c.$$

That is, there is at most a constant overhead in describing a string x with f versus describing x with g .

We would now like to choose a *minimal* universal description method U which minorizes every other potential description method, and then define a string’s complexity to be $C_U(x)$.¹

A priori, it is not clear that such a U should exist... And in fact, if we desire U to minorize *every* partial function g , then it does not! Here’s a quick sketch of the justification: pick an infinite sequence of strings $x_1, x_2, \dots$ with $p_1, p_2, \dots$ such that $C_U(x_i) = |p_i|$ for all i and such that $|p_i|$ is increasing. Pick a subsequence $p_{j_1}, p_{j_2}, \dots$ such that $|p_{j_i}| > 2|p_i|$ for every i , and define g as follows: $g(p_i) = f(p_{j_i})$ for all i , and g agrees with f everywhere else. Now, observe that

$$C_g(x_{j_i}) \leq |p_i| < |p_{j_i}|/2 = C_U(x_{j_i})/2.$$

Thus, for infinitely many strings z , $C_g(z) < C_U(z)/2$, so U does not minorize g .

¹Note that such a minimal U need not be unique, as two partial functions can minorize each other. But for the theory of Kolmogorov complexity, any such minimal U will suffice.

To remedy this, we restrict the class of valid description methods so that such a universal description method can emerge. The class of description methods will be all partial recursive functions, which are the set of partial functions for which there exists a Turing machine M which, on input p for which $f(p)$ is defined, halts with $f(p)$ written on the output tape. (Note that if $f(p)$ is not defined, then we make no promises about M 's behavior: in fact, it could be that M never terminates!)

Theorem 2. *There exists a partial recursive function U that minorizes every partial recursive function.*

Remark. Before diving into the proof, we make a brief comment about encoding pairs of strings. One can encode the pair $\langle x, y \rangle$ with $2|x| + |y| + 2$ bits: repeat each bit of x twice, add the string 01 as a sentinel, and then write the string y . This is not the most efficient method of encoding pairs of strings, but for our purposes this will suffice.

Proof. The description method U uses the Universal Turing machine. On input $\langle M, w \rangle$ (where we view M as the description of a TM and w as an input), U simulates M on input w and then outputs whatever M outputs (assuming it halts). We have

$$C_U(x) = \min\{|\langle M, w \rangle| : \text{TM } M \text{ halts on input } w \text{ and outputs } x\}.$$

We now demonstrate that U minorizes every other partial recursive function. Given any such function g , let M be a TM that computes it. Let x be any string, and let p be a shortest string such that $M(p) = x$, i.e., M halts on input p with x on its output tape. Clearly, $U(\langle M, p \rangle) = x$, and moreover $|\langle M, p \rangle| = c_M + |p|$, where $c_M = 2|\langle M \rangle| + 2$: importantly, c_M , is independent of p . The proof is therefore concluded by noting

$$C_U(x) \leq |\langle M, p \rangle| \leq c_M + |p| = c_M + C_g(x). \quad \square$$

Using this function U , we can finally define the Kolmogorov complexity.

Definition 3 (Kolmogorov Complexity). The *Kolmogorov complexity* of a string x is

$$K(x) := C_U(x) = \min\{|\langle M, w \rangle| : \text{TM } M \text{ halts on input } w \text{ and outputs } x\}.$$

We can use this function $K(x)$ to formally define what it means for a string to be “random”.

Definition 4 (Incompressible strings). A string x is *incompressible* or *Kolmogorov random* if $K(x) \geq |x|$.

Note that there exist incompressible strings of every length: there are 2^n binary strings of length n but only $2^n - 1$ binary strings of length strictly less than n .

2.3 Examples

1. The string 0^n has Kolmogorov complexity $\log n + O(1)$: we just need to specify n and a program which, on input n , outputs 0^n .
2. The first n (binary) digits of e also has Kolmogorov complexity $\log n + O(1)$, really for the same reason as above (use any program for approximating e via, say, its Taylor series).

3. Every string has $K(x) \leq |x| + O(1)$. (Justify!)

4. Do there exist strings x of length n with Kolmogorov complexity, say, $\sqrt{n}$? Yes! Take some $y \in \{0, 1\}^{\sqrt{n}}$ with $K(y) \geq \sqrt{n}$. Then, we claim that $x = y0^{n-\sqrt{n}}$ has Kolmogorov complexity $\sqrt{n} \pm \Theta(1)$. If it were too much shorter than $\sqrt{n}$, then we could describe y by describing x and then removing the last $n - \sqrt{n}$ 0's. On the other hand, here's a short way to describe x : describe y and then add an additional $n - \sqrt{n}$ 0's. This argument can be generalized to other (sublinear) complexities.

2.4 Undecidability of determining if a string is incompressible

First, we note that the set of strings that are *not* incompressible is recursively enumerable: on input x , consider all possible pairs $\langle M, w \rangle$ of length less than $|x|$, simulate M on w , and accept x iff some M outputs x on input w . Nonetheless, we have:

Theorem 5. *It is undecidable whether a string is incompressible.*

Note that this implies the set of incompressible strings is *not* recursively enumerable.

Proof. We'll provide two proofs. The first is very slick, while the second provides a reduction from the Halting Problem.

1. Assume for a contradiction we have a program P which can decide if a string is incompressible. Consider the program P' which, on input n , runs P on all strings of length n in lexicographical order and, as soon as P says that a string is incompressible, output that string. This gives a description of the lexicographically first incompressible string of length n of length $\langle P', \langle n \rangle \rangle = \log n + O(1)$, contradicting the fact that its Kolmogorov complexity is at least n .
2. Let $H = \{\langle M \rangle : \text{TM } M \text{ halts on input } 0\}$, and as before assume we have a program P which decides whether a given string is incompressible. We can use P to decide whether $\langle M \rangle \in H$ as follows. Let $n = |\langle M \rangle|$, and use P to decide for each y of length $2n$ whether or not it is incompressible. For each y that is compressible, find some program P_y that prints it out for which $|\langle P_y \rangle| < 2n$. Let t_y be the number of steps P_y requires to print out y , and let $t = \max_y t_y$. Then, run M on input 0 for t steps: if it halts, output “ $\langle M \rangle \in H$ ”; otherwise, output “ $\langle M \rangle \notin H$ ”.

Why does this work? It is clear that M does not halt on 0 then the above program will correctly say “ $\langle M \rangle \notin H$ ”. The only potential problem is that maybe M does halt on 0, but takes more than t steps to do so. So suppose for a contradiction that this is the case. Note that we can specify the running time of M on input 0 using only $n + O(1)$ bits, as we can just provide $\langle M \rangle$ plus a program that counts the number of steps. Thus, we can specify an upper bound t^* on the running time of all the P_y 's using only $n + O(1)$ bits. In this way, we can specify the set of all *compressible* strings of length $2n$ using only $n + 2\log(2n) + O(1)$ bits: give a description of t^* and the number $2n$, and then describe a program which simulates all programs of length less than $2n$ for at most t steps, and then outputs whatever the programs output. But then we can use this description of the set of all compressible

strings to describe the lexicographically first incompressible string using only $n + 2\log(2n) + O(1)$ bits! This gives the desired contradiction, as for large enough n , $2n > n + 2\log(2n) + O(1)$.

Thus, we have described a program that decides H , the Halting Problem. This is impossible, implying that there can be no such program P deciding if a string is incompressible.

□

3 Application: Prime Numbers

Kolmogorov complexity can be used to prove the infinitude of primes.

Theorem 6 (Euclid c. 300 BC). *There are infinitely many prime numbers.*

Proof. Suppose not. Let $p_1, p_2, \dots, p_k$ be a list of all the primes for some integer k . Let $m \in \mathbb{N}$ be a natural number whose encoding in binary is incompressible² and let $n = \lfloor \log_2 m \rfloor + 1$. Write m as a product of primes: for some non-negative integers $e_1, \dots, e_k$,

$$m = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}.$$

Note that we can describe m by providing $\langle e_1, e_2, \dots, e_k \rangle$. We claim that this is a short description of m , contradicting the incompressibility of m . Note that each $e_i \leq \log_2 m \leq n$, so it takes only $\log_2 n$ bits to describe each e_i . Hence, we can describe $\langle e_1, \dots, e_k \rangle$ with at most $(2k-1)\log_2 n + 2(k-1) + O(1) = O(\log_2 n)$ bits. (Note that k is constant, independent of n .) But, we assumed that $K(m) \geq n$. Assuming n is large enough (which we may, as there are incompressible strings of every length), this is a contradiction. □

²As a minor comment, note that the encoding in binary of such an integer m must begin with a 1. Thus, we need an incompressible string which begins with a 1. However, it cannot be the case that every compressible string of a given length begins with a 0. Do you see why?

Fast Integer Multiplication

Sai Sandeep

February 13, 2020

1 Introduction

Suppose that we are multiplying two integers - what is the algorithm that we use? The traditional algorithm that we learn in “grade school” is the following -

	30	+	7
20			
+	600		140
3	90		21

Table 1: Adding all the products, we get that that $23 \times 37 = 851$.

Even though it is excellent for pedagogical purposes, it has a problem - if the integers are large, then the algorithm is too slow. To multiply two n bit integers, the above algorithm requires $\Theta(n^2)$ operations. Can we do faster?

For a long time, it was believed that this might be tight. There was even a seminar organized in 1960 in Moscow to prove that integer multiplication (among other similar computational problems) requires $\Omega(n^2)$ operations. Surprisingly, during the seminar, Karatsuba came up with an algorithm that runs in $O(n^{\log_2 3}) \approx O(n^{1.58})$ time. The algorithm is based on a clever divide and conquer idea, similar to the Strassen algorithm for matrix multiplication. The next question is, can we do even faster?

It turns out that we can solve integer multiplication in $O(n \log n)$ time¹. This is achieved using fast polynomial multiplication algorithm.

2 Fast Polynomial multiplication

Suppose that we are given two polynomials $A(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$ and $B(x) = b_0 + b_1x + \dots + b_{n-1}x^{n-1}$. Can we find an efficient algorithm to compute the polynomial $P(x) = A(x)B(x)$? The naive algorithm that is similar to the grade school multiplication algorithm achieves $O(n^2)$ time complexity. Our goal is to find a faster algorithm.

Before delving into the algorithmic question, we first look into the problem of representing the polynomials. An obvious way to represent a polynomial is to write down all the coefficients. However, there is a different way to represent the polynomials: storing the evaluation of the polynomial at various points.

We first recall certain basic facts about polynomials:

Theorem 1. (Fundamental Theorem of Algebra) *A degree n polynomial with complex coefficients has exactly n complex roots.*

As a corollary, we get the following:

Corollary 1.1. *A degree n polynomial with coefficients from $\mathbb{C}$ is uniquely determined by its evaluation at $n+1$ points in $\mathbb{C}$.*

¹ $O(n \log n)$ complex arithmetic operations, to be precise.

Thus, we can also represent the polynomials $A(x)$ and $B(x)$ by their evaluations as $A(q_1), A(q_2), \dots, A(q_n), B(q_1), B(q_2), \dots, B(q_n)$. Note that this representation has a clear advantage: it is very easy to multiply the polynomials in this representation, just multiply the evaluations at respective points! But, this also has a disadvantage: evaluating the polynomial at a different point is hard, if we just store the evaluations at n points. This gives rise to the following question - can we efficiently transfer between the two representations? This way, we can be in best of the two worlds: first, convert the two polynomials A and B to their evaluation representation, multiply them in the evaluation representation, and finally convert back to the coefficient representation. A priori, this doesn't seem to be getting us any savings as the naive method of evaluating a degree n polynomial at n points requires $\Theta(n^2)$ time.

There is one final missing piece: we can choose the n points that we evaluate the polynomial as we wish. It turns out that if we choose the evaluation points to be the n th roots of unity, we can indeed evaluate the polynomials fast!

A small technical detail: henceforth, we assume that n is a power of 2. This is okay because we can achieve this by adding zero coefficients by at most doubling the degree.

Theorem 2. (Discrete Fourier Transform) Suppose that P is a degree $n-1$ polynomial. Then, we can evaluate P at all the n th roots of unity in total $O(n \log n)$ time.

Furthermore, we can also invert this operation efficiently:

Theorem 3. (Inverse Discrete Fourier Transform) Suppose that P is a degree $n-1$ polynomial. Given the evaluation of P at the n th roots of unity, we can compute all the coefficients of P in total $O(n \log n)$ time.

We first recall certain basic properties of complex numbers:

Definition 4. An n th root of unity is a complex number x such that $x^n = 1$.

Fact 5. There are n roots of unity, $1, \omega, \omega^2, \dots, \omega^{n-1}$, where $\omega = e^{\frac{2\pi i}{n}}$.

Proof. Note that Theorem 1 implies that there are exactly n complex numbers that are n th roots of unity. $(\omega^k)^n = e^{2\pi i k} = (e^{2\pi i})^k = 1^k = 1$. $\square$

Fact 6. The $\frac{n}{2}$ nd roots of unity are $1, \zeta, \zeta^2, \dots, \zeta^{\frac{n}{2}-1}$, where $\zeta = \omega^2$.

2.1 DFT

Let $A(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$ be the input polynomial. Our goal is to evaluate A at all the n th roots of unity. The idea is to use divide and conquer paradigm. First, we can break the polynomial A into A_{even} and A_{odd} , where $A_{\text{even}}(x) = a_0 + a_2x + \dots$, where $A_{\text{odd}}(x) = a_1 + a_3x + \dots$. We get $A(x) = A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2)$. Thus, we can recursively compute A_{even} and A_{odd} at all the $\frac{n}{2}$ nd roots of unity, and then use Fact 6 to evaluate the polynomial A .

Fast Fourier Transform (A, n)

1. Compute $A_{\text{even}}, A_{\text{odd}}$.
2. Compute Fast Fourier Transform ($A_{\text{even}}, \frac{n}{2}$), Fast Fourier Transform ($A_{\text{odd}}, \frac{n}{2}$).
3. For $0 \leq k < \frac{n}{2}$: $A(\omega^k) = A_{\text{even}}(\omega^{2k}) + \omega^k A_{\text{odd}}(\omega^{2k}) = A_{\text{even}}(\zeta^k) + \omega^k A_{\text{odd}}(\zeta^k)$.
4. For $\frac{n}{2} \leq k < n$: $A(\omega^k) = A_{\text{even}}(\omega^{2k}) + \omega^k A_{\text{odd}}(\omega^{2k}) = A_{\text{even}}(\zeta^{k-\frac{n}{2}}) + \omega^k A_{\text{odd}}(\zeta^{k-\frac{n}{2}})$.

The time complexity $T(n)$ is bounded as $T(n) = 2T(\frac{n}{2}) + O(n)$. By solving this recurrence, we obtain $T(n) = O(n \log n)$.

2.2 Inverse DFT

Our goal is to compute the unique polynomial A given the evaluations of A at $A(1), A(\omega), \dots, A(\omega^{n-1})$. Note that we have the following matrix equation:

$$\begin{bmatrix} A(1) \\ A(\omega) \\ \vdots \\ A(\omega^{n-1}) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \dots & \omega^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \dots & \omega^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

where the $n \times n$ matrix is denoted as the Fourier matrix F_n . To compute A from the evaluation, we need to find the inverse of the matrix F_n .

In fact, the inverse of the matrix F_n is the following:

$$G_n = \frac{1}{n} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \dots & \omega^{-(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{-(n-1)} & \omega^{-2(n-1)} & \dots & \omega^{-(n-1)(n-1)} \end{bmatrix}$$

Lemma 7. The above defined matrix satisfies $G_n F_n = I_n$.

Proof. The proof follows from the following observation:

$$\sum_{j=0}^{n-1} \omega^{kj} = \begin{cases} n, & \text{if } k \equiv 0 \pmod{n} \\ 0, & \text{otherwise} \end{cases}$$

The case when k is a multiple of n is trivial, as each summand on the left is equal to 1. When k is not a multiple of n , $x = \omega^k$ is an n th root of unity, and it satisfies $0 = x^n - 1 = (x-1)(1+x+x^2+\dots+x^{n-1})$. As $x \neq 1$, $\sum_{j=0}^{n-1} x^j = 0$. $\square$

Thus, in order to compute $G_n[A(1)A(\omega)\dots A(\omega^{n-1})]$, we repeat the same algorithm as DFT, except with ω replaced by $\frac{1}{\omega}$.

2.3 Final Algorithm

We describe the algorithm to multiply the two polynomials A and B .

Multiplying the polynomials A, B .

1. Let $1, \omega, \dots, \omega^{2n-1}$ be the $2n$ th roots of unity.
2. Compute $A(1), A(\omega), \dots, A(\omega^{2n-1})$ and $B(1), B(\omega), \dots, B(\omega^{2n-1})$ using the DFT algorithm.
3. Multiply them pointwise to compute $AB(1), AB(\omega), \dots, AB(\omega^{2n-1})$.
4. Using the inverse DFT algorithm, compute the polynomial AB .

The total running time of the above algorithm is $O(n \log n)$.

3 Fast Integer Multiplication

By using the fast polynomial multiplication algorithm, we can multiply two n bit integers using $O(n \log n)$ complex arithmetic operations, simply by representing the integers as polynomials with $x = 2$. Note that all the complex numbers involved can be bounded by polynomial in n . Thus, we can do any arithmetic operations of them in $\text{poly}(\log n)$ time. Thus, in time $O(n \text{poly}(\log n))$, we can multiply two integers. Schönhage and Strassen use these ideas together with a modification of FFT where they use a finite ring in stead of complex numbers, to achieve $O(n \log n \log \log n)$ time complexity. After a series of works, finally the time complexity $O(n \log n)$ was achieved last year by David Harvey and Joris van der Hoeven.

CS 252, Lecture 6: Spectral Graph Theory

Guest Lecture by Pravesh Kothari

1 Introduction

In this lecture, we give an overview of spectral graph theory, where in we use tools from Linear Algebra to study graphs. We demonstrate how we can “read off” combinatorial properties of graphs from their linear algebra properties. In particular, we focus on the problem of connectivity of graphs, and study how we can calculate the number of connected components of a graph from reading eigenvalues of its adjacency matrix.

We denote a graph G by $G = (V, E)$ where V is the set of vertices, and E is the set of edges. In this lecture, we assume all the graphs are undirected, unweighted, and there are no multiple edges or self loops. We also assume that the graphs that we study are d -regular i.e. all the vertices have degree d . Let $n = |V|$ denote the number of vertices in the graph.

We start with defining adjacency matrix and Laplacian of a graph.

Definition 1 (Adjacency matrix). Given a graph G , the adjacency matrix A_G is defined as the follows:

$$A_G(i, j) = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise.} \end{cases}$$

The Laplacian matrix of a graph is defined similar to the adjacency matrix, is easier to use and generalizes well to graphs that are not regular and weighted graphs.

Definition 2 (Laplacian of a graph). Given a graph G , the Laplacian matrix $L_{n \times n}$ is defined as $dI_{n \times n} - A$, where $I_{n \times n}$ is the $n \times n$ identity matrix. In other words, we have

$$L_G(i, j) = \begin{cases} d & \text{if } i = j \\ -1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise.} \end{cases}$$

2 Linear Algebra Background

Consider a real symmetric matrix $M \in \mathbb{R}^{n \times n}$ i.e. $M_{i,j} = M_{j,i}$ for all $i \neq j$.

Definition 3 (Eigenvalue, Eigenvectors). A scalar λ is called *eigenvalue* of M if there exist a vector $x \neq 0$ such that $Ax = \lambda x$. The corresponding vector x is called *eigenvector*.

Fact 4. The following are standard facts about eigenvalues of a real symmetric matrix M :

1. M has n real eigenvalues (including repetitions) $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$.
2. There exist eigenvectors $v_1, v_2, \dots, v_n$ such that $Mv_i = \lambda_i v_i$ for all $1 \leq i \leq n$, and the set of vectors $v_1, v_2, \dots, v_n$ are linearly independent. The eigenvectors corresponding to distinct eigenvalues are orthogonal.

Henceforth, we let $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ denote the eigenvalues of M . The corresponding eigenvectors are denoted by $v_1, v_2, \dots, v_n$. We assume that these n vectors are unit vectors, and are mutually orthogonal i.e. $\langle v_i, v_j \rangle = 0$ for all $i \neq j$.

Definition 5 (Quadratic form). The quadratic form of a matrix M is the function $M(v) = v^T M v$ that outputs a scalar on taking a vector from $\mathbb{R}^n$ as input.

Lemma 6. Let M be a real symmetric matrix. $v^T M v \geq 0$ for all vectors $v \in \mathbb{R}^n$ if and only if all the eigenvalues of M are non-negative.

Proof. Suppose that $v^T M v \geq 0$ for all $v \in \mathbb{R}^n$. Then, we claim that all the eigenvalues are non-negative. Suppose for contradiction that there is $\lambda < 0$ and a vector x such that $Mx = \lambda x$. We get $x^T M x = \lambda x^T x = \lambda(\sum_{i=1}^n x_i^2) < 0$, contradicting the fact that $v^T M v \geq 0$ for all vectors v .

Suppose that all the eigenvalues of M are non-negative. Then, for an arbitrary vector v , let $v = \alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_n v_n$. Such an expression is possible for every vector v since the vectors $v_1, v_2, \dots, v_n$ span the whole of $\mathbb{R}^n$. We have $v^T M v = \sum_{i=1}^n \lambda_i \langle v_i, v_i \rangle \geq 0$. $\square$

A real symmetric matrix with all eigenvalues being non-negative is called as a Positive Semidefinite matrix.

Lemma 7. (Rayleigh Coefficient) We have

$$\lambda_1 = \min_{v \neq 0} \frac{v^T M v}{v^T v}$$

and

$$\lambda_2 = \min_{v \neq 0, \langle v, v_1 \rangle = 0} \frac{v^T M v}{v^T v}$$

Proof. We only prove the first part, the proof of the second part follows very much along the same lines. As $\lambda_1 = \frac{v_1^T M v_1}{v_1^T v_1}$, we get $\lambda_1 \geq \min_{v \neq 0} \frac{v^T M v}{v^T v}$. Furthermore, let $v = \alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_n v_n$. We have $\frac{v^T M v}{v^T v} = \frac{\sum_{i=1}^n \alpha_i^2 \lambda_i}{\sum_{i=1}^n \alpha_i^2} \geq \lambda_1$ for all $v \neq 0$. $\square$

Recall the Laplacian of the graph G : L_G . For simplicity, we use L to mean L_G . For a vector $u = (u_1, u_2, \dots, u_n)$, we have

$$(Lu)_i = du_i - \sum_{j:(i,j) \in E} u_j = \sum_{j:(i,j) \in E} (u_i - u_j)$$

and

$$u^T Lu = \sum_{(i,j) \in E} (u_i - u_j)^2 \quad (1)$$

Using this, we can observe that L is Positive Semidefinite matrix.

When we project the vertices of G on to the real line such that the vertex i is mapped to u_i , the quadratic form $u^T Lu$ denotes the sum of squares of distances between neighboring vertices of G .

3 Graph Connectivity

Let G be an undirected unweighted d -regular graph and let L be the Laplacian matrix of G . Let $0 \leq \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ be the eigenvalues of L .

The smallest eigenvalue of L is always zero:

Lemma 8. $\lambda_1 = 0$.

Proof. As L is a positive semidefinite matrix, $\lambda_1 \geq 0$. Setting $u = (1, 1, \dots, 1)$, we have $Lu = (0, 0, \dots, 0)$, thus proving that 0 is an eigenvalue of L . $\square$

We can deduce if the graph G is connected or not by reading the next eigenvalue of L ! We prove the fact below:

Lemma 9. $\lambda_2 > 0$ if and only if the graph is connected. More generally, the number of zero eigenvalues of L is equal to the number of connected components of G .

Proof. Let k be the number of connected components of G and $S_1, S_2, \dots, S_k$ be the connected components of G .

First, we will prove that the number of zero eigenvalues is at least the number of connected components. Towards this, we define the following set of k vectors $u^{(1)}, u^{(2)}, \dots, u^{(k)}$ where $u^{(i)}$ is defined as follows:

$$u^{(i)}(j) = \begin{cases} 1, & \text{if } j \in S_i \\ 0, & \text{otherwise.} \end{cases}$$

Note the following:

1. $\langle u^{(i)}, u^{(j)} \rangle = 0$ for all $i \neq j$.
2. $Lu^{(i)} = 0$ for all $i \in \{1, 2, \dots, k\}$.

Thus, there are k mutually orthogonal vectors that are all eigenvectors of L corresponding to the eigenvalue 0. In other words, the number of zero eigenvalues of L is at least k .

Next, we will prove that the number of zero eigenvalues is at most k . From Equation (1), we can deduce that $u^T Lu = 0$ if and only if $u_i = u_j$ for all $(i, j) \in E$. That is, u should have equal value for all vertices in a connected component. Thus, if $Lu = 0$, then $u^T Lu = 0$, which implies that there exist

scalars $\alpha_1, \alpha_2, \dots, \alpha_k$ such that $u = \sum_{i=1}^k \alpha_i u^{(i)}$. This implies that every eigenvector of L corresponding to the eigenvalue 0 is contained in the subspace spanned by $\{u^{(1)}, u^{(2)}, \dots, u^{(k)}\}$. Thus, there are at most k linearly independent eigenvectors of L corresponding to the eigenvalue 0. This proves that the eigenvalue 0 of M is repeated at most k times. $\square$

4 Expansion of Graphs

We have seen that $\lambda_2 > 0$ if and only if the graph is connected. Can we say something more general? Does the magnitude of λ_2 have any meaning? It turns out that the absolute value of λ_2 indicates how much “robustly connected” G is. If λ_2 is large, the graph is “very” connected i.e. for every set $S \subseteq V$, a good fraction of edges originating in S cross S .

We first start with a few definitions.

Definition 10 (Cut/Boundary). For a set $S \subseteq V$, the *boundary* of S , denoted by δS is the number of edges that are adjacent to S that go out of S . That is,

$$\delta(S) = \sum_{i \in S} |\{j : (i, j) \in E\}|$$

Definition 11 (Conductance/Isoperimetric ratio of a set). For a set $S \subseteq V$, a scaled version of the fractional number of edges originating in S that cross it is called as the Conductance of S and denoted by $\Theta(S)$. To be precise,

$$\Theta(S) = \frac{\delta(S)}{d \frac{|S|}{n} |V \setminus S|}$$

Note that the above definition is within factor 2 of the fraction of the edges originating in S that go out of S .

Definition 12 (Conductance of a graph). The conductance of a graph G is defined as $\Theta(G) = \min_{S \subseteq V} \Theta(S)$.

The conductance of a graph is directly related to the second eigenvalue λ_2 of the Laplacian of the graph:

Theorem 13 (Cheeger’s inequality). $\frac{\lambda_2}{2d} \leq \Theta(G) \leq \sqrt{\frac{2\lambda_2}{d}}$.

Finally, we define Expander graphs, the graphs which are sparse, yet well connected.

Definition 14 (Expander Graphs). A sequence of d -regular graphs $G_1, G_2, \dots$ is called an Expander if there exist an absolute constant $C > 0$ such that $\frac{\lambda_2(G_i)}{d} \geq C$ for all i .

Expanders have various applications, both in theoretical computer science and beyond, ranging from fault tolerant networks to construction of error correcting codes.

CS 252, Lecture 7: Matchings & Determinants

1 Introduction

In this lecture, we use algebraic tools to study perfect matchings in graphs. We will focus on bipartite graphs. Let $H = (L, R, E)$ denote a bipartite graph where L and R are the partitions of the vertex set and $E \subseteq L \times R$ is the set of the edges of the graph. Given such a graph H , our goal is to find if there is a perfect matching i.e. whether there is a bijection $\sigma : L \rightarrow R$ such that for every $\ell \in L$, $(\ell, \sigma(\ell)) \in E$. There is a standard augmenting path approach to this problem, but we view this from an algebraic perspective.

Let $|L| = |R| = n$. We represent H by the $n \times n$ adjacency matrix M as follows: the rows of M are indexed by the elements of L and the columns are indexed by elements of R , and

$$M_{\ell,r} = \begin{cases} 1 & \text{if } (\ell, r) \in E \\ 0 & \text{otherwise.} \end{cases}$$

2 Relation to Determinant

A question then arises: what kind of pattern in the above adjacency matrix corresponds to the existence of perfect matching in H ?

This motivates the following definition:

$$P = \sum_{\sigma \in S_n} \prod_{\ell \in L} M_{\ell, \sigma(\ell)}$$

where S_n is the set of all the permutations of $\{1, 2, \dots, n\}$. The above defined quantity is called as *permanent* of the matrix M . Note that in our case, it precisely counts the number of perfect matchings in H . However, computing the permanent of a matrix is a very hard problem, from a computational perspective. $\square$

Thus, we look an alternative definition, the *determinant* of a square matrix M , denoted as $|M|$.

$$|M| = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{\ell \in L} M_{\ell, \sigma(\ell)}$$

¹It is #P complete. #P is a class of counting problems for which a single solution is easy to verify, for eg. given a matching, it is easy to verify whether it is a perfect matching or not. However, counting the number of solutions can be very hard, just like counting the number of perfect matchings of a bipartite graph is hard.

Here, $\text{sgn}(\sigma)$, the signature or sign of a permutation is equal to +1, if there is an even number of swaps we can perform to σ to reach identity permutation, and -1 otherwise.

There are several other ways to define the determinant as well, for eg. we can define it recursively as follows: The determinant of a matrix $P = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ is equal to $ad - bc$. For the 3×3 matrix, it is defined as follows:

$$Q = \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}$$

$$|Q| = a \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix}$$

Similarly, we can recursively define the determinant of a $n \times n$ matrix by using the definition of the determinant of a $(n-1) \times (n-1)$ matrix. Even though the signs seems rather arbitrary, determinant of matrices is very useful, and has a lot of applications. For eg., the determinant of a $n \times n$ square matrix is equal to the volume of the n dimensional parallelepiped spanned by the row (or column) vectors of the matrix. Thus, the determinant of a square matrix is non-zero if and only if the rows (or columns) of the matrix are linearly independent.

More importantly for us, a key distinction between the permanent and the determinant is that the determinant can indeed be computed very efficiently. We can use Gaussian elimination to transform the matrix into a lower triangular matrix, and then compute the determinant of the lower triangular matrix by simply multiplying the elements on the principal diagonal.

Thus, in order to check if H has a perfect matching or not, we can compute the determinant of the adjacency matrix M and check if it is equal to zero. Note that if there is no perfect matching in H , the determinant of M is equal to zero. However, the converse may not be true because of the signs involved in the definition of the determinant. It could be that there are several perfect matchings, and in computing the determinant with the signs, they could cancel each other out. One idea to fix this problem is to choose the elements of the adjacency matrix M cleverly. Note that as long as we assign non-edges to be zeroes, the determinant is zero if there is no perfect matching. Instead of assigning 1 to all the edges, we assign weights $W : E \rightarrow \{1, 2, \dots, T\}$ and redefine the new adjacency matrix M as follows $\square$

$$M_{\ell,r} = \begin{cases} 2^{W(\ell,r)} & \text{if } (\ell, r) \in E \\ 0 & \text{otherwise.} \end{cases}$$

Let the weight of a perfect matching $W(T) = \sum_{e \in E} W(e)$ be defined as the sum of the weights of its edges. The idea is to choose the weights cleverly such that one of the perfect matching stands out, thus making the determinant non-zero. In particular, we choose the weights W such that there is a unique perfect matching with minimum weight. We achieve this by choosing the weights W independently at random for each edge.

²We use the powers in 2 here to convert the products in determinant expression to sums of weights.

3 Isolation Lemma

The determinant of the redefined adjacency matrix is the following:

$$|M| = \sum_{\sigma} \begin{cases} 2^{\sum_{\ell \in L} W_{\ell, \sigma(\ell)}} & \text{if } \sigma \text{ is a perfect matching.} \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

Note that if there is no perfect matching, the value of $|M|$ is zero. We claim that if there is a unique perfect matching in H with minimum weight, then $|M|$ is non-zero.

Claim. Let $W : E \rightarrow \{1, \dots, T\}$ weights be assigned to the edges of the bipartite graph H , and M be as defined earlier. Suppose that there is a unique perfect matching in H with the least weight. Then, the determinant of M is non-zero.

Proof. Let $W_{\min}$ denote the minimum weight of a perfect matching in H . As all the weights assigned to the edges are integers and the perfect matching with minimum weight is unique, there is exactly one perfect matching with weight $W_{\min}$. From Equation (1), we can write the determinant of M as $|M| = \pm 2^{W_{\min}} + C \left(2^{W_{\min}+1} \right)$, for some constant C . This directly gives us that $|M| \neq 0$, as its remainder when divided by $2^{W_{\min}+1}$ is non-zero. $\square$

As alluded to earlier, we can get this unique perfect matching with minimum weight property by randomly assigning weights to each edge. In fact, we can set $T = 2m$, where $m = |E|$. At first look, this is surprising since there could be exponentially many perfect matchings, and thus there would be a lot of collisions i.e. many perfect matchings will have the same weight irrespective of how we assign the weights. However, the least weight perfect matching is unique most of the time.

Lemma 1. (Isolation Lemma)[[MVS77](#)] Let $\mathcal{F} \subseteq 2^{[m]}$ be a set family on a base set $E = [m] = \{1, 2, \dots, m\}$. Let $W : E \rightarrow \{1, 2, \dots, T\}$ be a weight function on E assigned uniformly and independently at random to each element of E . Then, with probability at least $1 - \frac{m}{T}$, there is a unique set in $\mathcal{F}$ with the smallest weight.

Proof. We will outline proof from [\[13-15\]](#).

We will prove a stronger statement that there is a unique minimum weight set is at least $(1 - \frac{1}{T})^m$ probability over assigning the weights. First, we assume that no set in $\mathcal{F}$ is a subset of other. If such a set pair exists, we can remove the larger set as it will never be a minimum weight set. Let $\mathcal{W}$ denote the set of all weight assignments $W : E \rightarrow \{1, 2, \dots, T\}$, and $\mathcal{W}_{>1}$ denote the set of weight assignments $W : E \rightarrow \{2, 3, \dots, T\}$. We define a function $\phi : \mathcal{W}_{>1} \rightarrow \mathcal{W}$ as follows: For an assignment of weights $W \rightarrow \{2, 3, \dots, T\}$, pick a set S_0 in $\mathcal{F}$ with minimum weight (as per W), and then let

$$W'(i) = \begin{cases} W(i) - 1 & \text{if } i \in S_0 \\ W(i) & \text{otherwise.} \end{cases}$$

We set $\phi(W) = W'$. We claim two things:

1. First, there is a unique set of minimum weight in $\mathcal{F}$ using the weights $\phi(W)$, for all $W \in \mathcal{W}_{>1}$. Note that S_0 is a set with minimum weight in W , and its weight reduced by $|S_0|$ from W to $\phi(W)$ and every other set's weight has reduced by at most $|S_0| - 1$ since we assumed that no set is a subset of other. Thus, we can infer that S_0 is the unique set with minimum weight as per the assignment of weights $\phi(W)$.
2. Second, for every $W_1 \neq W_2$ in $\mathcal{W}_{>0}$, $\phi(W_1) \neq \phi(W_2)$. If $\phi(W_1) = \phi(W_2)$, then according to the first claim, there is a unique set with minimum value in $\phi(W_1) = \phi(W_2)$. However, this implies that after adding 1 to the elements of that set, we get back both W_1 and W_2 , contradicting the fact that $W_1 \neq W_2$.

Combining the two claims, we can infer that there are at least $|\mathcal{W}_{>1}|$ set of assignment of weights in $\mathcal{W}$ that ensure that there is a unique minimum weight set. The proof follows then by observing that $|\mathcal{W}_{>1}| = (T - 1)^m$. $\square$

Now, set $T = 2m$. Using the isolation lemma, we can conclude that after assigning the weights randomly, with at least 50% probability, there is a unique perfect matching. Combining with Section 3, this implies that the determinant of M is non-zero with at least 50% probability, if there is a perfect matching in H . As before, if there is no perfect matching, the determinant of M is zero. We can combine with the following algorithm to check if H has a perfect matching:

Algorithm to find if H has a perfect matching

1. Set weights $W : E \rightarrow \{1, 2, \dots, 2m\}$ uniformly and independently at random.
2. Set the matrix M to be the adjacency matrix with weights 2^W .
3. Compute the determinant of M .
4. Return Yes iff the determinant is non-zero.

Note that the algorithm uses random assignment of the weights, and thus it could be wrong sometimes. In fact, the algorithm has an *one-sided* error. If H does not have a perfect matching, the algorithm always returns No. If H does have a perfect matching, the algorithm returns Yes with probability at least 50%. We can repeat the algorithm many times to reduce this error.

4 Polynomial Identity Testing

Instead of assignment weights at random, we can assign *indeterminates* to the adjacency matrix, and check if the resulting determinant polynomial is zero or not. To be more formal, we define the adjacency matrix M of the bipartite graph H as follows:

$$M_{\ell, r} = \begin{cases} x_{\ell, r} & \text{if } (\ell, r) \in E \\ 0 & \text{otherwise.} \end{cases}$$

We can observe that the determinant of M , when expanded as a polynomial, is equal to the zero polynomial if and only if H does not have a perfect matching. Thus, in order to check if H has a perfect matching or not, we can just check if this polynomial is equal to the zero polynomial or not. However there is a catch: this polynomial could have exponential number of terms! Thus, directly computing the coefficients of this polynomial is inefficient. We can instead evaluate the polynomial at an arbitrary point. Thus, given a polynomial as a “black box”, can we efficiently check if it is equal to the zero polynomial? This problem is known as Polynomial Identity Testing (PIT). There are fast randomized algorithms to do this. ³ Finding deterministic algorithms for this problem is a very big open problem in theoretical computer science!

Thus, we get a different randomized algorithm to find if H has a perfect matching. However, this begs the question: why do all this when there are simple deterministic algorithms using augmenting paths? A major reason is that the deterministic augment path algorithm is inherently sequential. These randomized algorithms can be easily parallelized. Furthermore, they also generalize well to other settings such as weighted and non-bipartite graphs.

References

- [MVV87] Ketan Mulmuley, Umesh V. Vazirani, and Vijay V. Vazirani. Matching is as easy as matrix inversion. *Combinatorica*, 7(1):105–113, 1987.
- [Ta-15] Noam Ta-Shma. A simple proof of the isolation lemma. *Electronic Colloquium on Computational Complexity (ECCC)*, 22:80, 2015.

³Randomized algorithms are algorithms that use random bits that may err sometimes, but succeed with good probability, like the one that we have seen in the previous section.

CS 252, Lecture 8: Constraint Satisfaction Problems

1 Introduction

A key endeavor in theoretical computer science is the following:

When and why do polynomial time algorithms exist? Can we explain the $\mathbf{P}$ vs $\mathbf{NP}$ completeness divide?

In general, characterizing problems for which there is a polynomial time algorithms turns out to be hard. And thus, we focus on structured class of problems to shed light on the above. To be specific, we are interested in the following question of dichotomy: is it true that every problem in our class is either in $\mathbf{P}$ or is $\mathbf{NP}$ hard?

Turns out that such a dichotomy does not exist in general. Ladner’s theorem states that if $\mathbf{P} \neq \mathbf{NP}$, then there is a language in $\mathbf{NP}$ that is neither in $\mathbf{P}$ nor is $\mathbf{NP}$ complete. We need the assumption of $\mathbf{P} \neq \mathbf{NP}$ since if $\mathbf{P} = \mathbf{NP}$, then every problem in $\mathbf{P}$ is also $\mathbf{NP}$ complete as well. However, the proof of Ladner is non-constructive and we are interested if this phenomenon occurs in “natural” problems. One such candidate is the Constraint Satisfaction Problems (CSP).

2 CSP

Informally, in an instance of a Constraint Satisfaction Problem, there are a collection of variables and a set of constraints on some subset of variables. Our goal is to determine if there is a way to assign values to the variables to satisfy all the constraints. We start with some examples:

1. 3-SAT: In the 3-SAT problem, we are given a set of variables $x_1, x_2, \dots, x_n$, and we have a set of constraints involving at most three variables. Each constraint is a disjunction of literals⁴. For example, the set of constraints $(x_1 \vee x_2 \vee \overline{x_4}), (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3}), (x_2 \vee x_3)$ form an instance of 3-SAT. The objective is to figure out if there is a assignment from $\{0, 1\}$ to the variables $x_1, x_2, \dots, x_n$ such that all the constraints can be satisfied. For example, in the above instance, assigning $x_1 = 1, x_2 = 1, x_3 = 0, x_4 = 0$ satisfies all the three constraints. (Here, we stick with the convention that 1 is True and 0 is False). 3-SAT is the canonical $\mathbf{NP}$ complete problem.
2. 1-in-3-SAT: Here, the constraints involve three literals and is satisfied if and only if exactly one of the three literals is true. For example, the problem instance could be $(\overline{x_1}, x_2, \overline{x_4}), (x_1, x_2, x_3), (x_2, x_3, x_4)$. There is no satisfying assignment to this instance. If $x_2 = 1$, the first constraint forces $x_1 = x_4 = 1$,

⁴A literal is either a variable x_i or its negation $\overline{x_i}$.

but then the second constraint has more than two true literals. If $x_2 = 0$, $x_4 = \overline{x_1}$, $x_3 = \overline{x_1}$ and $x_3 = \overline{x_4}$ all three of which can't hold simultaneously.

3. 2-SAT: It is similar to 3-SAT, except that here there are at most two literals in each constraint.
4. k -SAT: k -SAT is a generalization of 3-SAT in which there are at most k literals in each clause.
5. Horn-SAT: In the Horn-SAT problem, there is at most one positive literal (any any number of negated literals) in each clause.
6. Dual Horn-SAT: In the Dual Horn-SAT problem, there is at most negated literal (and any number of positive literals) in each clause.
7. NAE-3-SAT: Here, each constraint involves three literals, and is satisfied if and only if not all the literals have equal value i.e. there is at least one literal which is assigned true, and at least one literal which is assigned false. This problem makes sense even if we assume that no variable is negated in the constraints.
8. ODD-3-SAT: Here, each constraint involves three literals, and is satisfied if and only if there are odd number of literals that are assigned true.
9. So far we have considered only Boolean CSPs. We can also look at CSPs over non-Boolean domains. For example, if the domain is $\{0, 1, 2\}$ and the allowed constraints is (x, y) which is $x \neq y$, we get the classical graph three coloring problem.

Now, we formally define the Constraint Satisfaction Problems. For ease of presentation, we stick with the Boolean case.

Definition 1. (Constraint Template, Constraints) A constraint template $\Gamma = \{P_1, P_2, \dots, P_t\}$ is a set of predicates P_i s where each predicate P_i is a Boolean function $P_i : \{0, 1\}^{a_i} \rightarrow \{0, 1\}$ with arity a_i . A constraint over a constraint template Γ is an expression of the form $P(v_1, v_2, \dots, v_k)$ where $V = \{v_1, v_2, \dots, v_k\}$ is the set of the variables, and $P \in \Gamma$ is a predicate of arity k . The constraint is said to be satisfied by an assignment $\sigma : V \rightarrow \{0, 1\}$ if and only if $P(\sigma(v_1), \sigma(v_2), \dots, \sigma(v_k)) = 1$.

We assume that the set Γ and the arity of each predicate in Γ are finite.

Definition 2. (CSP(Γ)) An instance of CSP(Γ) is a set of variables V and a collection of constraints over Γ involving variables from V . Given such an instance, the objective is to determine if there is an assignment to V that satisfies all the constraints.

Note that the definition does not involve literals, and only variables. We can adjust the constraint template to account for this. For example, the 3-SAT is equal to CSP(Γ) with $\Gamma = \{P_0, P_1, P_2, P_3\}$ where $P_0(x, y, z) = x \vee y \vee z$, $P_1(x, y, z) = \overline{x} \vee y \vee z$, $P_2(x, y, z) = \overline{x} \vee \overline{y} \vee z$, $P_3(x, y, z) = \overline{x} \vee \overline{y} \vee \overline{z}$.

3 Dichotomy Theorem

If an instance of a CSP(Γ) is satisfiable, there is a short certificate to verify it: just the assignment of the variables that satisfies all the constraints. This proves that CSP(Γ) is in **NP** for every constraint template Γ . A natural question is the following: for which templates Γ , the problem CSP(Γ) can be solved in polynomial time? For which problem is it **NP** complete? Are there constraint templates for which it is neither?

This question is answered by the CSP Dichotomy Theorem.

Theorem 3. (CSP Dichotomy Theorem) For every constraint template Γ over a finite domain, CSP(Γ) is either in **P** or is **NP** complete.

The Boolean case of the above is proved by Schaefer in 1978, and the general case is proved independently by Bulatov and Zhuk in 2017.

We take a look back at the previous examples of CSPs:

1. 3-SAT: It is the canonical **NP** complete problem.
2. 1-in-3-SAT: It is **NP** complete as well.
3. 2-SAT can be solved in polynomial time by solving an auxiliary graph connectivity problem.
4. k -SAT: Since it generalized 3-SAT, it is **NP** complete too.
5. Horn-SAT and Dual Horn-SAT can be solved in polynomial time. It is a good exercise to come up with efficient algorithms to solve these problems.
6. NAE-3-SAT is **NP** complete. It is equivalent to asking whether a 3-uniform hypergraph² can be colored with 2 colors.
7. ODD-3-SAT can be solved in polynomial time using Gaussian elimination.
8. Graph 3-coloring is **NP** complete.

In light of the CSP dichotomy theorem, we are interesting in knowing which CSPs are solvable in polynomial time and which ones are **NP** complete. As we mentioned earlier, this question is answered by the Schaefer's theorem for the Boolean case.

Theorem 4. (Schaefer's theorem) For all Boolean templates Γ , CSP(Γ) is either in **P** or is **NP** complete. Furthermore, CSP(Γ) is in **P** if and only if at least one of the following holds:

1. All the predicates in Γ that are not constantly false are satisfied by assigning 1 to each of the variables.
2. All the predicates in Γ that are not constantly true are satisfied by assigning 0 to each of the variables.

²Hypergraphs are a generalized graphs, where each edge can connect more than two vertices.

3. Each predicate in Γ is equivalent to a conjunction of 2-SAT clauses.
4. Each predicate in Γ is equivalent to a conjunction of Horn-SAT clauses.
5. Each predicate in Γ is equivalent to a conjunction of Dual Horn-SAT clauses.
6. Each predicate in Γ is equivalent to a conjunction of linear equations.

4 Polymorphisms

The modern view of CSPs is taken through the lens of closure properties of the CSPs called as *polymorphisms*. Roughly speaking, polymorphisms are ways in we can combine satisfying assignments to a given CSP to obtain other satisfying assignments.

We start with 2-SAT as an example. Suppose that we have a 2-SAT instance I on n variables $x_1, x_2, \dots, x_n$. Let $u_1 \in \{0, 1\}^n$ be an assignment to the variables that satisfies all the constraints. Similarly, let u_2, u_3 be (not necessarily different from u_1) satisfying assignments to the given instance. Now, we define $v \in \{0, 1\}^n$ as follows: for every $i \in \{1, 2, \dots, n\}$, $v_i = \text{MAJ}_3((u_1)_i, (u_2)_i, (u_3)_i)$, where MAJ_3 is a 3-ary Boolean function that outputs 1 if and only if at least two of the inputs bits are equal to 1.

We claim that v is also a satisfying assignment to the 2-SAT instance I . In order to prove this, we need to prove that every clause in I is satisfied by the assignment v . Consider an arbitrary clause in I : Suppose it is $x_i \vee \bar{x}_j$. Note that (v_i, v_j) is actually equal to one of $((u_1)_i, (u_1)_j), ((u_2)_i, (u_2)_j), ((u_3)_i, (u_3)_j)$, as we are using the MAJ_3 function. Thus, v also satisfies the constraint $x_i \vee \bar{x}_j$ as each of u_1, u_2, u_3 satisfy it. As this argument holds for each clause in I , v is a satisfying assignment to the instance I . Thus, given three satisfying solutions to an instance of 2-SAT problem, we can take majority function on each bit to get a new satisfying solution to the same instance. This can be viewed as a discrete form of convexity³. There are efficient algorithms to optimize over convex sets in optimization, and similarly, in CSPs, if there are non-trivial polymorphisms, there are efficient algorithms to solve them.

We now formally define polymorphisms.

Definition 5. (Polymorphisms) Let Γ is a Boolean constraint template. A function $f : \{0, 1\}^m \rightarrow \{0, 1\}$ is said to be a polymorphism of Γ of arity m if the following holds: For every instance I of variables $V = \{v_1, v_2, \dots, v_n\}$ over Γ , and for every set of m satisfying assignments to I , $u_1, u_2, \dots, u_m \in \{0, 1\}^n$, the assignment

$$(f((u_1)_1, (u_2)_1, \dots, (u_m)_1), f((u_1)_2, (u_2)_2, \dots, (u_m)_2), \dots, f((u_1)_n, (u_2)_n, \dots, (u_m)_n))$$

also satisfies all the constraints in I .

There are some trivial polymorphisms for every constraint template Γ : simply output one of the original satisfying assignments i.e. $f(x_1, x_2, \dots, x_m) = x_i$ for some $i \in \{1, 2, \dots, m\}$. Such functions are known as dictators. For the 3-SAT problem, these are the only polymorphisms. Recall that 3-SAT is **NP** complete while 2-SAT is in **P**. In fact, this is not an accident, and this has been formalized in the modern CSP dichotomy theorem.

Theorem 6. (Modern CSP Dichotomy Theorem, Informal) For any constraint template Γ , $\text{CSP}(\Gamma)$ is in **P** if it has “interesting” polymorphisms. Else, it is **NP** complete.

We now take a look at the polymorphisms of CSPs in Schaefer’s theorem.

1. For the first case, the function that always outputs 1 is a polymorphism.
2. For the second case, the function that always outputs 0 is a polymorphism.
3. As we discussed earlier, MAJ of odd arities is a polymorphism of 2-SAT.
4. The binary $\text{AND}(\wedge)$ is a polymorphism of Horn-SAT.
5. The binary $\text{OR}(\vee)$ is a polymorphism of Dual Horn-SAT.
6. Parity, the sum of elements modulo 2, of odd arity is a polymorphism of linear equations.

³A set S is convex if for any two points $u, v \in S$, the line joining u and v is also in S .

CS 252, Lecture 9: Probabilistically Checkable Proofs and Hardness of Approximation

1 Introduction

In this lecture, we introduce the notion of Probabilistically Checkable Proofs (PCPs), and explain their connection to proving bounds on the existence of approximation algorithms.

Recall the complexity class **NP**: it is the set of languages whose membership can be efficiently certified. More formally, A language $L \subseteq \{0, 1\}^*$ is in the class **NP** if there exists a deterministic poly-time verifier V such that for every string $x \in \{0, 1\}^*$,

1. if $x \in L$, there exists a “proof” string y such that $|y|$ is $\text{poly}(|x|)$ and $V(x, y)$ accepts.
2. if $x \notin L$, then for every string y , $V(x, y)$ rejects.

For example, consider the problem 3-coloring of graphs. Here, given a string x containing the description of the graph G , if it is 3-colorable, we can certify this by simply presenting y to be the valid 3-coloring of the graph G . The verifier V simply reads x, y and checks if the coloring in y is a valid one for the graph G . Note that if the graph is not 3-colorable, the verifier will reject every proof string y . The first part, where we accept all the valid strings, is usually referred to as the *completeness* of the proof system. The second part, where we reject all the invalid strings, is instead called *soundness*.

An issue with the above proof system is that the verifier needs to read the proof entirely. Can we verify the proof without reading the whole proof? i.e. can we “spot check” the proof? A priori it may seem impossible - we might be able to encode an incorrect proof that has error only in one bit. For example, in the graph coloring case, even though the graph is not 3-colorable, there might be a 3-coloring that violates only one edge in the graph, and if we don’t read the colors assigned to every vertex, we might incorrectly accept the proof.

Furthermore, if we require the verifier to be a deterministic algorithm, checking fewer bits of the proof essentially means asking if there exists a short proof for every language in **NP**, which is unlikely. Thus, we restrict ourselves to randomized verifier. However, the previous issue remains: if we have a coloring that violates only one edge, we can’t find it by sampling only a few edges. Hence, the goal is to develop a notion of robust proof system such that true proofs are always accepted and for false proofs, there are bugs everywhere. The main question is the following: is there a proof system where we can query only a few bits in the proof, and decide whether it is correct or not? The famous PCP Theorem answers this question in affirmative.

Theorem 1. (PCP Theorem) *There exists an absolute constant q such that the following holds: for every language L in **NP**, there exists a randomized poly-time verifier V such that for every string $x \in \{0, 1\}^*$,*

1. *If $x \in L$, there exists y such that $|y|$ is $\text{poly}(|x|)$ such that $V(x, y)$ accepts.*
2. *If $x \notin L$, for every string y , $V(x, y)$ rejects with probability at least $\frac{1}{3}$.*

Furthermore, the verifier V only reads q (randomly chosen) bits in the proof.

Note that the error probability $\frac{1}{3}$ can be replaced with any constant < 1 as we can always amplify the probability by running the verifier algorithm multiple times. Also, the constant q is independent of $|x|$ or $|y|$! In fact, we can take q to be 3 in the above formulation i.e. to get $\frac{1}{3}$ rejection probability.

PCP Theorem is widely considered as one of the key results in theoretical computer science. It was originally proved in 1992 [AS92, ALM⁺92]. The original proof is quite involved, and a new insightful proof was presented in 2005 [Dim07]. Even though it seems to be a statement about just interactive proofs, it has widespread applications in many other areas. One of the main connections is in proving impossibility results on approximation algorithms.

2 Connection to hardness of approximation

Recall that 3-SAT is NP-complete. That is, given a satisfiable formula, no polynomial time algorithm can find a satisfying assignment. Can we find an approximation algorithm - one that given any satisfying formula, outputs an assignment that satisfies 99% of the clauses? Simply picking the assignment at random satisfies $\frac{7}{8}$ fraction of the constraints. However, it seems unlikely that there exists an algorithm that can find an assignment satisfying 99% of clauses. How do we prove such a hardness? Note that proving such a thing unconditionally would imply **P** $\neq$ **NP**, which we are quite far from proving. Instead, we prove that approximating these is **NP**-hard as well.

The way we achieve this is by so called “gap preserving” reductions. For example, we are aiming for the following type of reduction: given an instance C of CIRCUIT-SAT problem¹, we output a 3-SAT instance ϕ such that

1. If C is satisfiable, ϕ is satisfiable as well.
2. If C is not satisfiable, there is no assignment satisfying 99% of constraints in ϕ .

Recall that CIRCUIT-SAT problem is **NP**-complete. Thus, if we indeed have a reduction of the above kind, we can prove that finding an assignment satisfying 99% of a satisfiable 3-SAT instance is **NP**-hard. There is a very simple way to reduce CIRCUIT-SAT to a 3-SAT instance: for every gate, we assign a variable, and enforce the condition of the gate using 3-SAT clauses. For example, if we have an AND gate, and the two inputs are labeled as g_1, g_2 and the output is labeled as g_3 , we can write this as $(g_1 \vee g_2 \vee \bar{g}_3) \wedge (g_1 \vee \bar{g}_2 \vee \bar{g}_3) \wedge (\bar{g}_1 \vee g_2 \vee \bar{g}_3) \vee (\bar{g}_1 \vee \bar{g}_2 \vee g_3)$.

¹In the CIRCUIT-SAT problem, we are given a Boolean circuit containing AND, OR, NOT gates, and the goal is to determine if there is an assignment to the input gates that outputs True.

However, the above reduction may not be gap preserving i.e. if the input circuit C is not satisfiable, there is no guarantee on the number of clauses violated in ϕ . We only know that ϕ is not satisfiable. However, can we come up with a clever reduction that is in fact gap preserving? Turns out that this question is equivalent to the PCP Theorem!

Theorem 2. (*PCP Theorem, Inapproximability version*) *There is an absolute constant $c < 1$ such that the following holds: there is a polynomial time algorithm that outputs a 3-SAT formula ϕ given a Boolean circuit C such that*

1. *If C is satisfiable, ϕ is satisfiable as well.*
2. *If C is not satisfiable, no assignment to the variables can satisfy more than c fraction of constraints in ϕ .*

Theorem 2 is equivalent to Theorem 1. We only prove one direction of the implication in this lecture: suppose that Theorem 2 holds. We will prove that a probabilistically checkable proof for **NP** exists. First, since CIRCUIT-SAT is **NP**-complete, if we show the existence of probabilistic verifier for CIRCUIT-SAT, then we have shown the existence for every language in **NP**. (Why? - Do try this as an exercise.) Given an instance C of a CIRCUIT-SAT, our goal is to output a proof of satisfiability/unsatisfiability that can be verified by checking only a few bits. First, using Theorem 2 we find the 3-SAT formula ϕ and the proof string y in our proof system is the formula ϕ together with a purported satisfiable assignment σ to the variables in ϕ that satisfies all the clauses. The verifier then randomly picks a clause and checks if the assignment given satisfies it. We claim that it is a valid Probabilistically Checkable Proof system. Note that the verifier only reads constant many bits from the proof.

1. **Completeness:** Suppose that C is satisfiable. Then, ϕ is satisfiable as well, and thus the assignment σ satisfies all the clauses in ϕ . Thus, the Verifier outputs yes.
2. **Soundness:** Suppose that C is not satisfiable. Then, at least $1 - c$ fraction of clauses in ϕ are not satisfied in any assignment to the variables in ϕ . Thus, given any proof string y , the verifier rejects the proof with probability at least $1 - c$.

Now, we can repeat the same verifier algorithm many times, each time independently picking a random clause and checking if the purported assignment satisfies that clause. If we repeat t times this way, the probability that the verifier accepts an incorrect proof is c^t , which can be made as small a constant as possible, since c is a constant as well. The downside of this is that we increase the number of bits queried by the algorithm t times. But it is okay since in Theorem 1 we only claimed that we can read a constant number of bits to ensure that the probability of accepting an incorrect proof is at most $\frac{2}{3}$.

3 Applications

We prove a lot of problems are **NP**-hard by reducing 3-SAT to them. Some examples are graph coloring, maximum independent set in graph, knapsack etc. Similarly, once we have hardness of approximation for

3-SAT, we can use these reductions (and many new ideas!) to prove downstream hardness of approximation results. We demonstrate a concrete example here: Consider the reduction from 3-SAT to CLIQUE². We have a reduction that given a 3-SAT formula ϕ with m clauses, outputs a graph G such that:

1. If ϕ is satisfiable, then G has a clique of size m .
2. Else, there is no clique of size m in G .

If we inspect the reduction, we can actually infer that the maximum clique in G is equal to the maximum number of clauses satisfied by an assignment to the variables. From Theorem 1 we can in fact prove that not only is CLIQUE **NP**-hard, approximating the maximum CLIQUE is **NP** hard as well. That is, there exists a constant $c < 1$ such that given a graph G and a constant k , it is **NP**-hard to distinguish between the following:

1. **Completeness:** G has a clique of size k .
2. **Soundness:** There is no clique of size kc in G .

We can in fact amplify this to show that approximating CLIQUE to *any constant* is **NP**-hard.

References

- [ALM⁺92] Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. Proof verification and hardness of approximation problems. In *33rd Annual Symposium on Foundations of Computer Science, Pittsburgh, Pennsylvania, USA, 24-27 October 1992*, pages 14–23, 1992.
- [AS92] Sanjeev Arora and Shmuel Safra. Probabilistic checking of proofs; A new characterization of NP. In *33rd Annual Symposium on Foundations of Computer Science, Pittsburgh, Pennsylvania, USA, 24-27 October 1992*, pages 2–13, 1992.
- [Din07] Irit Dinur. The PCP theorem by gap amplification. *J. ACM*, 54(3):12, 2007.

²It is the same reduction done in 251. See [these slides](#)

CS 252, Lecture 10: The Probabilistic Method

1 Introduction

Consider the following puzzle: Suppose that 12% of earth's surface is land, and the rest is water¹. Irrespective of how the land and water is distributed on earth, it is always possible to inscribe a cube in earth such that all the vertices of the cube are on water!

We can prove this as follows: inscribe a cube inside earth uniformly at random i.e. rotate the cube randomly. Now, the probability that a given vertex of the cube lies on land is exactly equal to 0.12. Let the random variable X denote the number of points of the cube that lie on land. Let X_i , $1 \leq i \leq 8$ denote the indicator variable equal to 1 if the i th vertex of the cube lies on land, and equal to 0 otherwise. We have $\mathbb{E}[X_i] = \Pr(X_i = 1) = 0.12$. By linearity of expectation, we have $\mathbb{E}[X] = \sum_{i=1}^8 \mathbb{E}[X_i] = 0.96 < 1$. That is, the average number of vertices of the cube on land is less than 1. Since X is an integer random variable, it implies that there exists a random configuration in which the number of vertices of the cube on land is equal to 0. Intuitively, this is true because not every one can be above average. More formally, we can use the Markov's inequality:

Lemma 1. (Markov's Inequality) Let X be a non-negative random variable. For every $a > 0$, we have

$$\Pr(X \geq a) \leq \frac{\mathbb{E}[X]}{a}$$

Thus, $\Pr(X > 0.97) < 1$, or in other words, $\Pr(X = 0) > 0$. Hence, there exists a rotation of the cube such that all the vertices of the cube are on water.

We can actually directly also prove that $\Pr(X = 0) > 0$ as follows:

$$\begin{aligned} \Pr(X = 0) &= 1 - \Pr(X \geq 1) \\ &= 1 - \Pr(\exists i : X_i \geq 1) \\ &\geq 1 - \sum_i \Pr(X_i \geq 1) = 1 - 0.96 > 0 \end{aligned}$$

where in the last step, we have used the union bound.

Essentially, what we have done is the following: when we are asked to show the existence of a cube with all the vertices on water, instead of directly showing it, we have constructed a probability space over

the possible cubes and have shown that there is a non-zero probability that a cube satisfies our required property. It may seem that we are doing unnecessary work here: why compute the probability of a cube having all vertices on water? why not directly show the existence of the cube with all the vertices on water, given a land water scenario? The issue is that showing the cube exists "constructively" is very difficult. Instead, we come up with a nice probability space where we can easily compute the probability of our required configuration, and then show that it's non-zero, and we are done!

This way of showing an existence of an object by constructing a clever probability space is known as the *probabilistic method*. It is one of the most powerful tools in Combinatorics and Theoretical Computer Science. It is also very general, applicable in many varied scenarios. In this lecture, we discuss three applications of it: first, in Ramsey theory, where the method first originated, and then second, MAX-CUT, a well studied problem in Computer Science, and then finally an application in Extremal Combinatorics.

2 Ramsey Theory

We start with a new puzzle: In any group of six people, there are either three mutual acquaintances, or three mutual non-acquaintances. Or in graph theoretic terms, in any graph on 6 vertices, there is either a clique of size 3 or an independent set of size 3. An alternate view is in terms of two coloring complete graphs: suppose that we color each edge of K_6 with either red or blue, then there is a monochromatic triangle.

We can prove it as follows: consider vertex 1, and look at the edges adjacent to 1. Since there are five edges, at least three of them should have the same color. Without loss of generality, let $(1, 2), (1, 3), (1, 4)$ be colored red. Now, consider the triangle $(2, 3, 4)$. If any of the edges in this triangle is colored red, the corresponding edge together with 1 forms a red triangle. If all the edges of the triangle are colored blue, we of course have a monochromatic triangle.

Note that the above is tight: the graph C_5 , the cycle on five vertices does not have any clique or independent set of size 3. We can generalize this to arbitrary k as follows:

Definition 2. (Ramsey Number) For an integer $k \geq 3$, the Ramsey number $R(k)$ is defined as the smallest integer n such that any graph on n vertices contains either a clique or an independent set of size k .

We have proved above that $R(3) = 6$. $R(4)$ is proved to be equal to 19. We can also show that $R(k) \leq \binom{2k-1}{k-1}$ by an inductive argument. Note that asymptotically, it is equivalent to $R(k) \leq O\left(\frac{4^k}{\sqrt{k}}\right)$ ².

What about lower bounds? In order to show $R(k) > m$, we need to show that there exists a graph on m vertices without a k sized clique or independent set. One construction that we can come up with is the following: consider a disjoint set of $k-1$ blocks $V_1, V_2, \dots, V_{k-1}$ each containing $k-1$ vertices. For every pair of vertices u, v , they are adjacent if and only if they are in different blocks. There is no clique or independent set of size k in this graph. Thus, $R(k) \geq \Omega(k^2)$. For some time, it was conjectured that this is optimal. Then, Erdős gave a striking proof of $R(k)$ being exponential in k , introducing the probabilistic method.

²Note that $\binom{n}{\frac{n}{2}} \approx \frac{2^n}{\sqrt{n}}$.

¹Even though this is incorrect now, it could have been the case at some point in the distant past.

Theorem 3. (Erdős, 1947) $R(k) \geq \Omega(2^{\frac{k}{2}}k)$.

Proof. We will use the probabilistic method. The first goal is to construct a probability space on graphs on n vertices. The most natural way to do this is the following, known as Erdős-Renyi random graph $G(n, \frac{1}{2})$: for each pair (i, j) such that $1 \leq i < j \leq n$, independently add an edge between (i, j) with probability equal to $\frac{1}{2}$. Let $G = (V, E)$ denote the random graph sampled according to this distribution. Let $S \subseteq V$ be a subset of vertices of size k . The probability that S is a clique or an independent set is equal to $2^{1-\binom{k}{2}}$. Thus, by union bound, the probability that there exists a clique or an independent set of size k in G is at most

$$\binom{n}{k} 2^{1-\binom{k}{2}} \leq 2 \left(\frac{ne}{k} \right)^k 2^{-\binom{k}{2}} \leq \frac{1}{3}$$

where we set n to be $k2^{\frac{k}{2}} \frac{1}{10e}$. Hence, the probability that the graph does not have a k -clique or k -independent set is non-zero. Thus, there exists a graph with $\Omega(2^{\frac{k}{2}}k)$ vertices that does not have any clique or independent set of size k . $\square$

A couple remarks: The idea of probabilistic method is also used (implicitly) by another great mind Claude Shannon around the same time in 1948 to prove the existence of good error correcting codes. The study of random graphs that we used, picking each edge independently, is a rich field on its own, with many applications.

The above construction shows that there exists a graph on n vertices that does not have a clique or independent size of size roughly $2 \log n$. One might wonder if there is a way to construct such a graph efficiently. Note that we can always brute force over all the graphs of a given size to find the graph that we need, but we are interested if we can find one such graph in time polynomial in n . And surprisingly, this is a major open problem! It is not just a mathematical puzzle, it has connections to so called “randomness extractors”, which deal with outputting perfectly random bits from biased sources. There has been a great amount of progress on this problem in the last 3 – 4 years, culminating in the construction of graphs on n vertices with no clique or independent set of size $(\log n)^{\log \log n}$.

3 MAX-CUT

In the MAX-CUT problem, we are given a graph $G = (V, E)$, and the goal is to find a cut $S \subseteq V$ that maximizes edges that cross S i.e. $E(S, \bar{S})$. Computing the MAX-CUT of a given graph is a well studied NP-complete problem. However, in every graph, there is a cut for which at least half the edges cross. We will prove this using the probabilistic method.

Theorem 4. For every graph G , there is a cut for which half the edges in G cross.

Proof. As before, we take a random cut and prove that the probability that half the edges cross it is non-zero. To be precise, we take each vertex independently with probability $\frac{1}{2}$ to S . Let X be the random

variable denoting the number of edges crossing the cut S . For an edge $e = (u, v)$, X_e be the indicator random variable equalling 1 if S cuts e , and 0 otherwise. We have

$$\mathbb{E}[X_e] = \Pr(u \in S \wedge v \notin S) + \Pr(u \notin S \wedge v \in S) = \frac{1}{2}$$

By linearity of expectation, we have $\mathbb{E}[X] = \sum_{e \in E} \mathbb{E}[X_e] = \frac{|E|}{2}$. Thus, by Markov’s inequality, $X \geq \frac{|E|}{2}$ with non-zero probability. $\square$

Note that this gives a trivial randomized algorithm to approximate the MAX-CUT of a graph with approximation ratio $\frac{1}{2}$. This was the best algorithm for a long time until in 1994 when Goemans and Williamson gave an algorithm using semidefinite programming that achieves 0.878 approximation factor.

4 Applications in Extremal Combinatorics

What are the maximum possible number of edges in a graph without any triangles? What are the maximum possible number of subsets of $\{1, 2, \dots, n\}$ where in no subset is contained in other? In Extremal Combinatorics, we study these questions like these, on the extremal properties of combinatorial objects.

Now, we demonstrate that probabilistic method can also be useful to “prove” theorems, not just showing the existence of certain objects. Let $\mathcal{F}$ be a family of subsets of $\{1, 2, \dots, n\}$. $\mathcal{F}$ is said to be an antichain if no set in $\mathcal{F}$ is contained in a different set in $\mathcal{F}$. How large can $\mathcal{F}$ be?

Theorem 5. (Sperner’s theorem) The size of any antichain on $\{1, 2, \dots, n\}$ is at most $\binom{n}{\lfloor \frac{n}{2} \rfloor}$.

Proof. At first, it seems that there is no intuitive probability space associated with the problem. But we can consider the following inspired space: Let Ω be the probability space of uniformly random orderings of $\{1, 2, \dots, n\}$. For a set $S \in \mathcal{F}$, let E_S be the event that in the random ordering, the union of first $|S|$ elements is equal to S . Note that

$$\Pr(E_S) = \frac{|S|!(n - |S|)!}{n!} = \frac{1}{\binom{n}{|S|}}.$$

Furthermore, since for every pair $S, T \in \mathcal{F}$, neither is contained in other, we can deduce that $\Pr(E_S \cap E_T) = 0$. Thus, by union bound, we get that

$$\sum_{S \in \mathcal{F}} \frac{1}{\binom{n}{|S|}} \leq 1.$$

Since for every j , $\binom{n}{j} \leq \binom{n}{\lfloor \frac{n}{2} \rfloor}$, we get that

$$|\mathcal{F}| \frac{1}{\binom{n}{\lfloor \frac{n}{2} \rfloor}} \leq 1$$

which proves our required claim. $\square$

Note that the above bound is in fact tight: we can just consider all the subsets of $\{1, 2, \dots, n\}$ that are of size equal to $\lfloor \frac{n}{2} \rfloor$.

CS 252, Lecture 11: Expander Graphs

1 Introduction

Expander graphs are sparse yet highly connected graphs. That is, for every subset S of vertices of the graph, there are a lot of edges leaving S . In other words, we cannot disconnect the graph into near-equal pieces by cutting a few edges. This notion is useful in various contexts ranging from design of computer networks to proving the existence of probabilistically checkable proofs.

There are many ways to define an expander graph formally: in terms of spectral expansion, vertex expansion or edge expansion. We can also define them separately for bipartite and general graphs. However, in this lecture, we will stick with the spectral notion of expanders. We will also consider general graphs. This connects well with random walks and pseudorandom properties of the expanders.

Recall from the spectral graph theory lecture that the adjacency matrix A of a d -regular graph $G = (V, E)$ with n vertices has n eigenvalues $d = \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. We have also proved that the graph is connected if and only if $\lambda_2 < d$. Let $\lambda(G) = \max\{|\lambda_2|, |\lambda_n|\}$. Note that¹

$$\lambda(G) = \max_{x: \langle x, u \rangle = 0} \frac{\|Ax\|}{\|x\|}$$

where $u = \left(\frac{1}{n}, \frac{1}{n}, \dots, \frac{1}{n}\right)$. Recall that we have proved earlier that $\lambda(G) \leq d$.

Definition 1. (Expander graphs) A graph $G = (V, E)$ is said to be an (n, d, λ) -expander if $|V| = n$, each vertex of the graph has degree d , and $\lambda(G) = \lambda$.

The parameter $d - \lambda(G)$ is known as the *spectral expansion* of the graph G . As we alluded to earlier, a different way to define expander graphs is in terms of *edge expansion*:

$$\min_{S \subseteq V, |S| \leq \frac{n}{2}} \frac{E(S, \bar{S})}{|S|}$$

Both the notions of expansion are actually closely related. We have

$$\frac{E(S, \bar{S})}{|S|} \geq \frac{d - \lambda_2}{2} \forall S : |S| \leq \frac{n}{2}.$$

¹Unless stated otherwise, $\|\cdot\|$ denotes the ℓ_2 norm.

The converse of this is also true. Graphs with good edge expansion also have good spectral expansion. This is given by the Cheeger inequality:

$$\exists S : \frac{E(S, \bar{S})}{|S|} \leq O\left(\sqrt{d(d - \lambda_2)}\right)$$

2 Construction of expander graphs

Of course expander graphs have a lot of useful properties, but we really need to construct them so that we can make use of them. By constructing expander graphs, we mean constructing a family of d -regular graphs $G_1, G_2, \dots$, such that G_i has i vertices. The goal is to find a polynomial time algorithm that takes input n, d and outputs the graph G_n in the family.

Random graphs are good expanders. In fact, one can view expander graphs as *pseudorandom* in the sense that they have a lot of properties of random graphs. Random d -regular graphs actually achieve $\lambda_2 \leq O(\sqrt{d})$. This is roughly, the best expansion that we can get. Such graphs, whose $\lambda_2 = 2\sqrt{d-1}$ are known as Ramanujan graphs.

Even though random graphs are good expanders with high probability, finding explicit construction of expander graphs is a challenging question. There are various known such explicit constructions:

1. (Discrete torus expanders) The first known explicit construction of expanders is due to Margulis, Gabber and Galil, where $G = (V, E)$ with the vertex set $V = \mathbb{Z}_m^2$, and a vertex (x, y) is adjacent to the following vertices: $(x \pm y, y), (x \pm (y + 1), y), (x, y \pm x), (x, y \pm (x + 1))$. It is a 8-regular graph. We can prove that λ_2 of this graph is $8c$ for $c < 1$.
2. (p -cycle with inverse chords) The graph $G = (V, E)$ with vertex set $V = \mathbb{Z}_p$, and edges connect each node x with $x + 1, x - 1, x^{-1}$. The arithmetic is $\text{mod } p$, and set $0^{-1} = 0$.

3 Pseudorandom properties of expander graphs

A key property of expander graphs is the so called expander mixing lemma. In a random graph with constant degree d , the number of edges between two sets S and T is roughly equal to $\frac{d|S||T|}{n}$. Expander graphs also mimic this property:

Theorem 2. (Expander Mixing Lemma) Let G be an (n, d, λ) -expander. Then, for any two subsets S, T of vertices of the graph,

$$\left| E(S, T) - \frac{d|S||T|}{n} \right| \leq \lambda \sqrt{|S||T|}$$

Proof. We will estimate the value of $(\mathbf{1}_S)^T A \mathbf{1}_T$, where $\mathbf{1}_S$ is the indicator vector of the set S :

$$(\mathbf{1}_S)_j = \begin{cases} 0 & \text{if } j \notin S \\ 1 & \text{if } j \in S. \end{cases}$$

$\mathbf{1}_T$ is also defined analogously. As before, let $u = \left(\frac{1}{n}, \frac{1}{n}, \dots, \frac{1}{n}\right)$. Let $\mathbf{1}_S = |S|u + v_1$, $\mathbf{1}_T = |T|u + v_2$. Note that $\langle v_1, u \rangle = \langle v_2, u \rangle = 0$. Furthermore, we get

$$\|v_1\| = \sqrt{(n - |S|) \frac{1}{n^2} + |S| \left(1 + n^2 - \frac{2}{n}\right)} = \sqrt{\frac{1}{n} + |S| - \frac{2|S|}{n}} \leq \sqrt{|S|}$$

Similarly, $\|v_2\| \leq \sqrt{|T|}$.

The number of edges between S and T is equal to $(\mathbf{1}_S)^T A \mathbf{1}_T$. We can write it as

$$\begin{aligned} &= (|S|u^T + v_1^T) A (|T|u + v_2) \\ &= (|S|u^T + v_1^T) (d|T|u + Av_2) \\ &= \frac{d|S||T|}{n} + d|T|v_1^T u + |S|u^T Av_2 + v_1^T Av_2 \\ &= \frac{d|S||T|}{n} + v_1^T Av_2 \quad (\text{since } \langle v_1, u \rangle = 0, u^T Av_2 = (Au)^T v_2 = d\langle u, v_2 \rangle = 0) \end{aligned}$$

Thus, we get

$$\begin{aligned} \left| E(S, T) - \frac{d|S||T|}{n} \right| &= v_1^T Av_2 \\ &\leq \|v_1\| \|Av_2\| \quad (\text{By Cauchy Schwartz}) \\ &\leq \|v_1\| \lambda \|v_2\| \quad (\text{since } \langle v_2, u \rangle = 0) \\ &\leq \lambda \sqrt{|S|} \sqrt{|T|} \end{aligned}$$

□

A couple remarks:

1. Note that the above theorem is applicable even if S and T intersect. In fact, both S and T can be the same set as well: in this case, we get a good estimate on the number of edges present inside a given set.
2. If we set $S = T$ to be an ϵ sized set, we can infer that most of the edges adjacent to S cross the set S . This is a very useful expansion property.

4 Application of expanders to derandomization

Suppose that there is a randomized algorithm for a language L using n bits such that

1. If a string $x \in L$, then the algorithm accepts with probability 1.
2. If a string $x \notin L$, then the algorithm rejects with probability at least $\frac{1}{2}$.

Our goal is to reduce the error probability $\frac{1}{2}$ of the algorithm. A natural idea is to repeat the algorithm multiple times. Suppose that we repeat the algorithm t times. Then, the error probability goes down to $\frac{1}{2^t}$, which is great, but the number of random bits used by the algorithm is equal to nt . Can we get the same exponential reduction in error probability, but with smaller amount of randomness?

The idea is to cleverly reuse the randomness by making “correlated” choices, rather than picking each set of bits independently. One way to achieve this is by performing a random walk on an expander graph. Let G be an expander on 2^n vertices, and with degree d . We first start at a random vertex v in G , using n random bits. Next, we pick a random neighbor of v , say v_1 . We need $\log d = O(1)$ bits for this. Next, we continue the random walk from v_1 . We repeat this process till we pick t vertices overall. The overall random bits that we needed is equal to $n + O(t)$.

Note that each vertex in the graph corresponds to a set of n bits. We run the algorithm using all the t sets of random bits obtained by the random walk. If the algorithm outputs No for any of these choices of random bits, we output No, else we output Yes. We still accept all the strings $x \in L$ with probability 1. How do we argue about the probability that we accept incorrect strings? In order to do so, we need an estimate on the probability that a t -step random walk on expander graph stays inside a set of size $\frac{n}{2}$ through out the walk. Turns out this probability is also exponentially small in t ! Using this, we obtained a “randomness efficient” algorithm that uses $n + O(t)$ random bits that still gets error probability exponentially small in t .

Now we prove the above mentioned bound:

Theorem 3. (Random walk on expander graph) Let $G = (V, E)$ be an (n, d, λ) -expander. Let $B \subseteq V$ be a subset of vertices such that $|B| = (1 - \delta)n$. Then, the probability that a random walk $V_1, V_2, \dots, V_t$ of $t - 1$ steps starting at a uniformly random vertex of G completely stays inside B is

$$\Pr(V_i \in B \forall i \in [t]) \leq \left(1 - \delta \left(1 - \frac{\lambda}{d}\right)\right)^{t-1}$$

Proof. Let P be the projection matrix that zeroes out all the indices not in B . It is a diagonal matrix such that

$$P_{i,j} = \begin{cases} 1 & \text{if } i = j, i \in B \\ 0 & \text{otherwise.} \end{cases}$$

Let $A' = \frac{A}{d}$ be the random walk matrix of the graph G i.e. for a probability distribution μ on the vertices, $A'\mu$ gives the probability distribution after taking a single random walk step. Thus, the probability that all the t vertices of the random walk are in B is equal to

$$\Pr(V_i \in B \forall i \in [t]) = \|PA'PA' \dots PA'Pu\|_1$$

Since $P^2 = P$, we can write the above as

$$\|(PA'P)^{t-1}Pu\|_1 \leq \sqrt{n} \|(PA'P)^{t-1}Pu\|$$

where we have used Cauchy-Schwartz inequality to relate ℓ_1 and ℓ_2 norms of a vector in $\mathbb{R}^n$. We now bound the largest absolute value of eigenvalues of $PA'P$: Let $\nu = \max_{\|x\|=1} x^T PA'P x$. Let $y = Px$.

We have $\nu = y^T A' y$. As before, we write $y = y^\parallel + y^\perp$, where $y^\parallel = \alpha u$, and $\langle y^\perp, y^\parallel \rangle = 0$. Note that $\|y^\parallel\|^2 + \|y^\perp\|^2 = \|y\|^2 \leq \|x\|^2 = 1$. Let $\mathbf{1} = (1, 1, \dots, 1)$. We have $y^\parallel = \alpha u$, where $\alpha = \langle y, \mathbf{1} \rangle$. Thus, $\|y^\parallel\|^2 = \frac{1}{n} \langle y, \mathbf{1} \rangle^2 = \frac{1}{n} \langle y, P\mathbf{1} \rangle^2 \leq \frac{1}{n} \|y\|^2 \|P\mathbf{1}\|^2 = (1 - \delta) \|y\|^2$.

We now bound ν :

$$\begin{aligned}
\nu &= y^T A' y \\
&= \left((y^\parallel)^T + (y^\perp)^T \right) A' (y^\parallel + y^\perp) \\
&\leq \|y^\parallel\|^2 + \frac{\lambda}{d} \|y^\perp\|^2 \\
&= \|y^\parallel\|^2 + \frac{\lambda}{d} \left(\|y\|^2 - \|y^\parallel\|^2 \right) \\
&= \left(1 - \frac{\lambda}{d} \right) \|y^\parallel\|^2 + \frac{\lambda}{d} \|y\|^2 \\
&\leq \|y\|^2 \left(\left(1 - \frac{\lambda}{d} \right) (1 - \delta) + \frac{\lambda}{d} \right) \\
&= \|y\|^2 \left(1 - \delta \left(1 - \frac{\lambda}{d} \right) \right) \\
&\leq 1 - \delta \left(1 - \frac{\lambda}{d} \right)
\end{aligned}$$

Substituting it in the above probability, we get

$$\begin{aligned}
\Pr(V_i \in B \ \forall i \in [t]) &\leq \sqrt{n} \left\| (P A' P)^{t-1} P u \right\| \\
&\leq \sqrt{n} \left(1 - \delta \left(1 - \frac{\lambda}{d} \right) \right)^{t-1} \|P u\| \\
&= \sqrt{n} \sqrt{\frac{1 - \delta}{n}} \left(1 - \delta \left(1 - \frac{\lambda}{d} \right) \right)^{t-1} \\
&\leq \left(1 - \delta \left(1 - \frac{\lambda}{d} \right) \right)^{t-1}
\end{aligned}$$

□

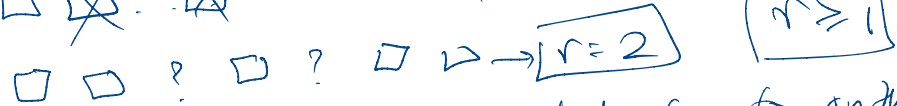
Lecture 12 - Reed-Solomon Coding

Today we will describe a classical, and widely used and popular method to cope with errors that occur in storage/communication of digital data
- a glimpse into the rich field of CODING THEORY

Send packets on a channel that can drop/erase up to r packets



We know the indices of the erased pkts



How can you send one packet of info on this channel?

Answer: Easy - replicate pkt $(r+1)$ times and send all copies



Can also see this is the best possible

Ok, what if you want to send k pkts at once?

Naive repetition scheme: $k(r+1)$ pkts.

Factor $(r+1)$ redundancy.

Q: Can one do better?

Yes, by "coding" pkts together $r=1$



Best soln for any k, r ?

Observe: Need to send at least $(k+r)$ pkts
i.e. add at least r redundant pkts



erase r of them



Remarkably, there is a simple scheme that achieves redundancy r

k pkts $\mapsto (k+r)$ pkts

s.t. any k of the received pkts suffice to recover the original k pkts

OPTIMAL!! How? Algebra/polynomials

Assume pkts are elements in $\{0, 1, 2, \dots, q-1\} = \mathbb{F}_q$

(q prime)

$\mathbb{F}_q$ is a "field" with addition & multiplication & subtraction & division (by $\neq 0$)

Example fields: $\mathbb{R}$ (reals), $\mathbb{Q}$ (rationals), $\mathbb{C}$ (complex no.), modulo q

Only "non-obvious" fact: inverses of nonzero els. exist modulo a prime q .

i.e. $a \neq 0 \quad \exists b$ s.t. $ab \equiv 1 \pmod{q}$
(q prime)

Polynomials over a field $\mathbb{F}$
 Expression of form

Non
zero
polynomial

$$P(X) = a_0 + a_1 X + a_2 X^2 + \dots + a_d X^d \quad (d = \text{degree})$$

(a_d ≠ 0)

Also 0 is a polynomial
 $1 + 2X + 4X^3$ → Coefficients $a_i \in \mathbb{F}$

Evaluate Poly $P(X)$ at pt. $\alpha \in \mathbb{F}$

$$P(\alpha) = a_0 + a_1 \alpha + a_2 \alpha^2 + \dots + a_d \alpha^d \in \mathbb{F}$$

α is a root of $P(X)$ if $P(\alpha) = 0$
 (equivalently $(X - \alpha)$ divides $P(X)$)

A fundamental theorem: A degree d (nonzero) polynomial over any field $\mathbb{F}$ has at most d roots

Proof is not hard, by induction on degree

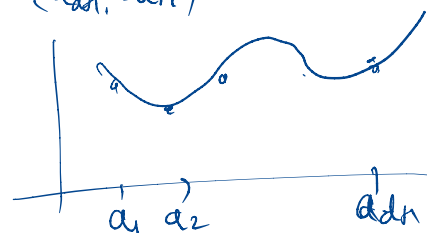
Using the "division with remainder" property of polys over a field $\mathbb{F}$

$$A(X) = \underbrace{Q(X)}_{\text{quotient}} B(X) + \underbrace{R(X)}_{\text{remainder}} \quad \deg(R(X)) < \deg(B(X))$$

Theorem: Let $a_1, a_2, \dots, a_{d+1} \in \mathbb{F}$ be distinct and $b_1, b_2, \dots, b_{d+1} \in \mathbb{F}$ be arbitrary.

There is a unique polynomial Q of degree $\leq d$ s.t.
 $Q(a_i) = b_i$ for $i = 1, 2, \dots, d+1$

(a₁, b₁)
(a₂, b₂)
⋮
(a_{d+1}, b_{d+1})



Proof:

Existence: Lagrange interpolation ✓

Uniqueness: Follows from the fundamental thm.

Suppose Q_1, Q_2 both explain the data
 $\tilde{Q} := Q_1 - Q_2 \quad \tilde{Q}(a_i) = 0$ for $i = 1, \dots, d+1$
 $\deg(\tilde{Q}) \leq d$ (of $\deg \leq d$) $\Rightarrow \tilde{Q} = 0$
 $\Rightarrow Q_1 = Q_2 \quad \square$

Back to "erasure" correction

$(P_0, P_1, \dots, P_{k-1})$ are the k pks ($P_i \in \mathbb{F}_2[x]$)
 Pick $a_1, a_2, \dots, a_{k+r} \in \mathbb{F}$, a_i distinct

$$P(X) := P_0 + P_1 X + P_2 X^2 + \dots + P_{k-1} X^{k-1} \quad (\deg \leq k-1)$$

[Reed-Solomon Coding]

$$(P_0, P_1, \dots, P_{k-1}) \mapsto (P(a_1), P(a_2), \dots, P(a_{k+r}))$$

Encoding $\equiv$ polynomial evaluation [can also speed up using Fast Fourier Transform]
 (1960)

$$P(a_1) P(a_2) \dots P(a_{k+r})$$

↓ r erasure

$$P(a_1) \quad ? \quad P(a_2) \quad P(a_3) \quad ? \quad \dots \quad ? \quad P(a_{\text{env}})$$
$$\text{data: } (a_i, p(a_i)) \quad L \text{ is unbounded}$$

Use theorem to find the unique $d_{\text{deg}} < k$ poly that interpolates this data.

Measure recovery $\equiv$ Polynomial interpolation

Comment:

Comment:

$P_1 P_2 \mapsto P_1 P_2 P_1 P_2$

Original pkts
appear as k of 16
coded pkts

Above scheme doesn't have this feature
Exer: How will you modify the encoding
 to have this property & still guarantee
 tolerance to r erasures?

What about p2os that are corrupted (and that goes undetected)
Error-correction?

Error-correcting: $(p_0, p_1 \dots p_{k-1}) \mapsto \langle p(a_1), p(a_2) \dots p(a_{k+r}) \rangle$
| encoder

$y_i \neq P(a_i)$ for upto ϵ indices i

 $y_1 \quad y_2 \quad \dots \quad y_{K+r}$

Key: You don't know where the errors are!
Would like to correct them & recover $P(X)$

Lemma: If $e \leq \lfloor \frac{r}{2} \rfloor$, then $P(X)$ uniquely
~~is~~ identifiable from the noisy evaluations
 (y_i^j)

Pr $\rightarrow$ If two poly PEQ 8th differ from
in $\leq$ places $\rightarrow (ds < k \text{ poly})$
Poly (Pail)

$P(a_i) \neq y_i$ for $\leq e$ val. of $i \Rightarrow P(a_i) \neq Q(a_i)$
 $Q(a_i) = y_i$ — " ———— for at most $2e$ val. of i

$$2e \leq r \Rightarrow P(a_i) = Q(a_i) \text{ for } \geq k \text{ values of } a_i$$
$$\Rightarrow P = Q$$

Challenge: Find $P(x)$ efficiently

Approach: To locate the errors
(also find $P(x)$ together with that)

Error-locator polynomial: $E(x) := \prod_{i: \sigma_i \neq 0} (x - a_i)$

$i = \text{Plat} \neq y_i$
(the p that's uniquely identifiable)

Observation

Define $Q(X, Y) := (Y - P(X)) E(X)$

Note: $Q(a_i, y_i) = (y_i - P(a_i)) E(a_i) = 0$

Idea of algo

① Forget $N(X)$ factor
as $P(X) \in X$ and simply
find $\tilde{E}(X) \neq 0$, $\tilde{N}(X)$ s.t.
 $\deg \tilde{E} \leq e$ $\deg \tilde{N} \leq e + k - 1$
s.t. $\forall i, \tilde{E}(a_i) y_i - \tilde{N}(a_i) = 0$

② If $\frac{\tilde{N}(X)}{\tilde{E}(X)}$ is a
 $\deg(k-1)$ poly, output it.

We don't know $Q(X, Y)$

$$Q(X, Y) = E(X) Y - P(X) E(X)$$

$$= E(X) Y - \tilde{N}(X)$$

$$\deg(E) \leq e = \lfloor \frac{r}{2} \rfloor$$

$$\deg(N) \leq e + k - 1$$

→ This is a
linear system
in coeffs of $\tilde{E}$
& $\tilde{N}$.

Cruz: Any $\tilde{N}, \tilde{E}$ output by
Step ① must satisfy

$$\tilde{N}(X) = \tilde{E}(X) P(X)$$

if $P(a_i) \neq y_i$ for at most
 $e = \lfloor \frac{r}{2} \rfloor$ vals of a_i

[This part not covered during lecture but
included as notes]

Two things to establish about algorithm:

- Efficiency
- Correctness

Efficiency: Step 2 is easy, just polynomial division
Step 1 amounts to finding a nonzero solution to a
homogeneous linear system with unknowns
being coefficients of $\tilde{E}$ & $\tilde{N}$. Can be done
in polytime using Gaussian elimination

CORRECTNESS

① A solution $\tilde{E}(X), \tilde{N}(X)$ subject to stipulated
degree constraints exists. $\tilde{E}(X) = E(X)$ (the error
locator poly)
- Pf: Indeed can take $\tilde{E}(X) = E(X)$
and $\tilde{N}(X) = E(X) P(X)$

② If $P(a_i) \neq y_i$ for at most $e = \lfloor \frac{r}{2} \rfloor$ locations,
then any $\tilde{N}$ & $\tilde{E}$ found in Step 1
must satisfy $\tilde{N}(X) = \tilde{E}(X) P(X)$
[So Step 2 correctly outputs $P(X)$]

Proof: Observe that $\tilde{N}(a_i) - \tilde{E}(a_i) P(a_i) = 0$
for every i s.t. $P(a_i) = y_i$

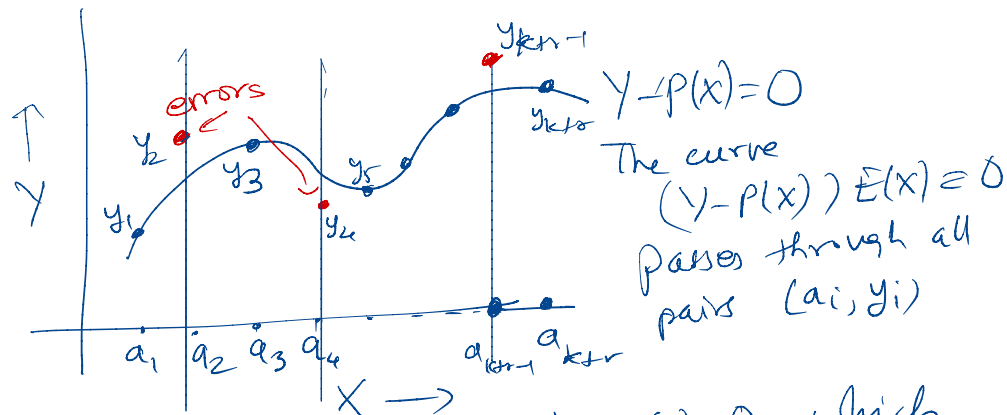
$$\text{Define } R(X) := \tilde{N}(X) - \tilde{E}(X) P(X)$$

- degree of $R \leq e + k - 1 = k + \lfloor \frac{r}{2} \rfloor - 1$
- R has $\geq k + r - e$ roots (all pts a_i s.t. $P(a_i) = y_i$)
 $= k + r - \lfloor \frac{r}{2} \rfloor = k + \lceil \frac{r}{2} \rceil$

Thus $R(x)$ has more roots than its degree
 $\Rightarrow R(x) = 0 \Rightarrow \tilde{N}(x) = \tilde{E}(x) P(x)$
 as desired $\square$

Geometric view

$$E(x) = (x - a_2)(x - a_4)(x - a_{k+r})$$



The correct curve $Y - P(x) = 0$, which explains a lot ($\geq k + \lceil \frac{r}{2} \rceil$) of the picture "emerges" as a factor in the picture when we interpolate a curve $Q(x, y) = 0$ (with specific degree restrictions) through all the pairs $(a_i, y_i), i=1, 2, \dots, k+r$.

CS 252, Lecture 13: Pseudorandomness: Hardness vs Randomness

1 Introduction

So far in the course, we have seen many useful randomized algorithms. All of them need uniformly random bits—how do we obtain these random bits? There are various practical ways to do this, such as using a last digit of a high precision computer clock, using a function of CPU clocks etc. An issue with these methods is that there is no way to certify the random bits that we obtain. They often have correlations among the bits, which can impact the correctness or running time of the randomized algorithms. In this lecture, we look at this issue and study how to obtain good random bits from perfectly random bits.

Generating good random bits is essential to cryptography as well. For example, one time padding requires a random string of length equal to the message length, and this random string cannot be reused. Can we somehow reduce this randomness requirement? In general, can we generate more random bits from a small number of random bits? From an information theory perspective, Shannon proved that such an absolute function $f: \{0, 1\}^s \rightarrow \{0, 1\}^n$ cannot generate uniformly random bits (or even some approximation of it) from a uniformly random s bits. Intuitively, the "entropy" of uniform distribution on s bits is less than that on $n > s$ bits, and no function can increase entropy by itself.

Instead, we study this problem from a computational perspective: what if we want to generate $n > s$ bits that are not necessarily uniformly random, but look random to any adversary with limited computational power? This key concept is called as "pseudorandomness" where in we calibrate a distribution as uniform from an observer's view or put in other words, "randomness in the eye of the beholder".

2 Computational Indistinguishability

Suppose that D_1 and D_2 are probability distributions on $\{0, 1\}^n$. We can view them as for every $x \in \{0, 1\}^n$, $D_1(x)$ (and respectively $D_2(x)$) denote the probability of x in the distribution D_1 (and respectively D_2). We say that D_1 and D_2 are statistically ϵ -close if for every function $T: \{0, 1\}^n \rightarrow \{0, 1\}$,

$$|\Pr_{x \sim D_1}[T(x) = 1] - \Pr_{x \sim D_2}[T(x) = 1]| \leq \epsilon$$

Definition 1. (Total Variation Distance) For a pair of distributions D_1, D_2 on $\{0, 1\}^n$, we define the total variation distance $\Delta_{TV}(D_1, D_2)$ as the maximum distinguishability between the distributions over all possible functions $T: \{0, 1\}^n \rightarrow \{0, 1\}$.

$$\Delta_{TV}(D_1, D_2) = \max_T |\Pr_{x \sim D_1}[T(x) = 1] - \Pr_{x \sim D_2}[T(x) = 1]|$$

Exercise. Prove that $\Delta_{TV}(D_1, D_2) = \frac{1}{2} \|D_1 - D_2\|_1 = \frac{1}{2} \sum_{x \in \{0,1\}^n} |D_1(x) - D_2(x)|$.

Note that for any pair of probability distributions D_1 and D_2 , $\Delta_{TV}(D_1, D_2) \in [0, 1]$.

Now, we formally define computational indistinguishability.

Definition 2. (Computational Indistinguishability) A pair of distributions D_1 and D_2 on $\{0, 1\}^n$ are said to be (C, ϵ) -computationally indistinguishable if for every machine A of complexity $C(n)$ cannot distinguish between D_1 and D_2 better than ϵ i.e.

$$|\Pr_{x \sim D_1}[A(x) = 1] - \Pr_{x \sim D_2}[A(x) = 1]| \leq \epsilon$$

In the definition, we have intentionally used a vague definition for “machine” and “complexity” since it can be instantiated with more concrete settings such as Turing machines with complexity being the running time, Boolean circuits with complexity being the size of the circuits etc.

A distribution D on $\{0, 1\}^n$ is said to be (C, ϵ) -pseudorandom if any machine with complexity $C(n)$ is fed with this distribution instead of the uniform distribution, the acceptance probability changes by at most ϵ .

Definition 3. $((C, \epsilon)$ -pseudorandom) A distribution D over $\{0, 1\}^n$ is said to be (C, ϵ) -pseudorandom if the uniform distribution $\mathcal{U}_n$ on $\{0, 1\}^n$ and D are (C, ϵ) -computationally indistinguishable.

3 Pseudorandom generators

How to produce pseudorandom distributions? The starting point is usually a small number of perfectly random bits, known as *seed*.

Definition 4. (Pseudorandom generators) A function $G : \{0, 1\}^{s(n)} \rightarrow \{0, 1\}^n$ is said to be (C, ϵ) -pseudorandom generator (PRG) with seed length $s(n)$ if $G(\mathcal{U}_k)$ is (C, ϵ) -pseudorandom.

Informally, a PRG stretches a perfectly random string into a longer “random looking” string. As we have mentioned earlier, obtaining a PRG that fools all possible functions without any limitations on the complexity $C(n)$ is statistically impossible, even with $s = n - 1$. Computationally however, it is possible, for eg, with respect to Boolean circuits, polynomial time algorithm etc. We generally seek $s(n) \ll n$, such as $s(n) = n^\epsilon$ or even $O(\log n)$.

The key motivation for defining PRGs is to use them to derandomize randomized algorithms. We first start with a very small seed, generate a longer random string that we require and use it instead of uniform bits for the algorithm. Since the algorithm cannot distinguish this random distribution from the uniform distribution, the algorithm still works fine. Since we are using much smaller number of random bits, we can just iterate over all possible choices for the seed, and pick the majority result. This gives us a deterministic algorithm that in fact is as good as the original randomized algorithm. We formalize this in the exercise below:

Exercise. (Derandomizing from PRG) If G is a $(C(n), \frac{1}{10})$ -PRG with seed length s and can be computed in $2^{O(s)}$ time, then a randomized algorithm with time complexity $C(n)$ and success probability $\frac{2}{3}$ can be derandomized to run in time $2^{O(s)}C(n)$.

4 Hardness vs Randomness

Suppose that we want to construct a pseudorandom generator that expands stretch by just a bit i.e. $G : \{0, 1\}^s \rightarrow \{0, 1\}^{s+1}$. We can achieve this as follows: given an input $x \in \{0, 1\}^s$, we output $s + 1$ bits $(x, g(x))$ where $g : \{0, 1\}^s \rightarrow \{0, 1\}$ is a “hard” function. Here, hardness of g is in terms of *average case* hardness. Being average case hard for a computational class of complexity $C(n)$ and a small constant ϵ means that for every machine A in this computational class, A cannot compute g even slightly better than random guessing i.e. $\Pr_{x \in \{0,1\}^s}(A(x) = g(x)) \leq \frac{1}{2} + \epsilon$. We contrast this average case hardness with worst case hardness: in the traditional worst case hardness, we say g is hard if every A in the computational class cannot compute g on some input, but in the average case hardness, such an A cannot compute g even on average i.e. just doing better than random guessing over all the input strings is hard.

Now, given such a hard function g , we claim that the output of $G : x \in \{0, 1\}^s \rightarrow (x, g(x)) \in \{0, 1\}^{s+1}$ is (C, ϵ) -pseudorandom. Intuitively, the reasoning is as follows: the first s bits are anyway uniformly random, and for the class C , the last bit appears to be uniform given the first s bits as well. This is discussed more formally in the homework.

The above argument is a key link between “hardness” and “randomness”: we have shown how to use an average case hard function to obtain pseudorandom distributions. But there is a catch: if g is supposed to be very hard, how can G itself compute g in the first place? There are two possible avenues to fix this issue:

1. Allow G more time than the tests it fools. This is okay for derandomization, but is not useful for cryptographic purposes.
2. Modify the above PRG construction to create asymmetry that gives G an edge compared to the tests it fools.

We now look at the first avenue in more detail.

4.1 Nisan Wigderson PRG Construction

Even though allowing more time for the PRG is a good idea for derandomization purposes, there is still another issue with the above construction: we are increasing the stretch by only a single bit. One could modify the above construction to output two extra bits by simply partitioning the input into two parts, and applying an average case function g on the first $\frac{s}{2}$ bits, and then similarly applying g on the last $\frac{s}{2}$ bits, and concatenating the resulting two bits with the original s bits. We can extend this to output k bits by partitioning $\{1, 2, \dots, s\}$ into k parts and applying an average case hard function to each part. However, this method cannot get our output beyond $2s$ bits. In order to do so, we need to use the average case hard function more cleverly, and the way we do this is by applying it to a carefully chosen subsets of $\{1, 2, \dots, s\}$.

In the Nisan-Wigderson PRG construction, we pick subsets $S_1, S_2, \dots, S_n \in 2^{[s]}$ of cardinality m each. Note that we do not want these subsets to intersect in many places since the bits will no longer be

¹The set $\{1, 2, \dots, s\}$ is often represented as $[s]$.

pseudorandom. We achieve this by picking subsets such that for every $i, j \in [n]$, $|S_i \cap S_j| \leq l \leq \frac{m}{100}$. Such a set system of subsets from $[s]$, where all the sets have cardinality m , and any pair of subsets intersect in at most l elements is known as a (s, m, l) -combinatorial design. They are well studied objects, and for the above setting of parameters, we have set families with $n = 2^{\Omega(s)}$ exponential number of subsets.

In fact, we can relate these combinatorial designs with the error correcting codes that we have studied in the previous lecture: first represent every subset $S_i \subseteq [s]$ by $\{0, 1\}^s$ with 1s in locations $j : j \in S_i$ and 0s elsewhere. Now, we are looking at picking a large number of vectors in $\{0, 1\}^s$ such that each of them have exactly m 1s, and every pair of them differ in at least $2(m - l)$ bits. This is precisely a question of picking codes with large distance, and large rate which we can achieve, both using randomized constructions and also efficient explicit concatenation techniques.

After picking these subsets, the fact that any pair of subsets intersect in small number of elements suffices to argue about the pseudorandomness of the output. Note that the output of this approach is exponentially larger than the input size s . Furthermore, there are “worst case to average case” hardness amplification results where in we prove the existence of average case hard function using the existence of a worst case hard function. Combining this with the Nisan Wigderson construction, we can actually prove that if there are problems that can be solved in exponential time $2^{O(n)}$ but also require circuits of size $2^{\Omega(n)}$ (which is widely believed to be true) then every randomized algorithm running in polynomial time can be derandomized to run in **P**. This is a strong evidence to the fact that in fact, randomness might not make a difference for polynomial time algorithms.

Now, we take a look at a different approach to obtaining PRGs.

4.2 Using one way permutations

Previously, we had $G(x) = (x, g(x))$ where g is a hard function to compute. In stead, we can keep an easy function g but modify the input x so that there is no way to recover it from the scrambled output. To be precise, we set $G(x) = (\pi(x), h(x))$ where $\pi : \{0, 1\}^s \rightarrow \{0, 1\}^s$ permutes the input bits. (We interchangeably use π as a permutation $\pi : [s] \rightarrow [s]$ and also as a function on s Boolean inputs that outputs s bits.) The property that we require of π is that it should be a *one-way* permutation: it is easy to apply, and hard to invert i.e. given $y \in \{0, 1\}^s$, it is hard to find $x \in \{0, 1\}^s$ such that $\pi(x) = y$. In fact, we also need that it is hard to predict $h(x)$ given $y = \pi(x) \in \{0, 1\}^s$. Such a Boolean function $h : \{0, 1\}^s \rightarrow \{0, 1\}$ is called as a *hardcore predicate* for the one way permutation π . Note that $h(x)$ being hard to predict given y implies that $(y, h(x))$ is a pseudorandom distribution.

We now discuss some examples of hardcore predicates:

1. The least significant bit (lsb) function is a hardcore predicate for the RSA function $\pi(x) = x^e \bmod N$.
2. The function $\text{half}_{p-1}(x) = \begin{cases} 1, & \text{if } x \in \{0, 1, \dots, \frac{p-1}{2}\} \\ 0 & \text{otherwise.} \end{cases}$ is a hardcore predicate for the modular exponentiation function $\pi(x) = g^x \bmod p$ for some prime p . For this example, lsb function is not a hardcore predicate.

Going from one extra bit to polynomially many extra bits can be done in a straightforward manner: we first start with $x \in \{0, 1\}^s$, compute $x_1 = \pi(x)$, $b_1 = h(x)$, and then $x_2 = \pi(x_1)$, $b_2 = h(x_1)$ and so on. Finally, we can append x with all the b_i s to obtain a pseudorandom distribution. This helps in expanding a stretch of n^ϵ bits to obtain n pseudorandom bits.

Wouldn't the sentence "I want to put a hyphen between the words Fish and And and And and Chips in my Fish-And-Chips sign," have been clearer if quotation marks had been placed before Fish, and between Fish and and, and and and And, and And and and, and and and And, and And and and, and and and Chips, as well as after Chips?⁴

— Martin Gardner, *Aha! Insight* (1978)

BMO: I finished with my computations! As I thought, it's a real language they are speaking based on intonations! [speaking to Chips and Ice Cream] Chips? Chips chips chips! Ice cream?

Chips: Chips chips chips!

Ice Cream: Ice cream ice cream!

— "Chips and Ice Cream", *Adventure Time* season 6, episode 34 (April 30, 2015)

For every polynomial-time algorithm you have,
there is an exponential algorithm that I would rather run.

— Alan Perlis (first Turing Award winner)

B

Faster Exponential Algorithms

[Read Chapters 2, 3, and 12 first.]

Status: Unfinished.

There are many useful problems—including but not limited to the set of all NP-hard problems—for which the fastest algorithm known runs in exponential time. Moreover, there is *very* strong evidence (but alas, no proof) that it is *impossible* to solve these problems in less than exponential time—it's not that we're all stupid; the problems really are that hard!

But sometimes we have to solve these problems anyway. In these circumstances, it is important to ask: *Which* exponential? An algorithm that runs in $O(2^n)$ time, while still

⁴If you ever decide to read this sentence out loud, be sure to pause briefly between 'Fish and and' and 'and and and And', 'and and and And' and 'and And and and', 'and And and and' and 'and and and And', 'and and and And' and 'and And and and', and 'and And and and' and 'and and and Chips'!

Did you notice the punctuation I carefully inserted between 'Fish and and' and 'and', 'and' and 'and and And', 'and and and And' and 'and and and And', 'and and and And' and 'and', 'and' and 'and And and and', 'and And and and' and 'and And and and', 'and And and and' and 'and', 'and' and 'and and and And', 'and and and And' and 'and and and And', 'and and and And' and 'and', 'and' and 'and And and and', 'and And and and' and 'and', 'and' and 'and And and and', 'and And and and' and 'and', 'and' and 'and and and And' and Chips?

unusable for large instances, is still significantly better than an algorithm that runs in $O(4^n)$ time; an algorithm that runs in $O(1.5^n)$ or $O(1.25^n)$ is better still.

The most straightforward method to find an optimal solution to an NP-hard problem to recursively generate *all* possible solutions via recursive backtracking and check each one: all satisfying assignments, or all vertex colorings, or all subsets, or all permutations, or whatever. However, many (perhaps even most) NP-hard problems have some additional structure that allows us to prune away most of the branches of the recursion tree, thereby drastically reducing the running time.

B.1 3SAT

Let's consider the mother of all NP-hard problems: 3SAT. Given a boolean formula in conjunctive normal form, with at most three literals in each clause, our task is to determine whether any assignment of values of the variables makes the formula true. Yes, this problem is NP-hard, which means that a polynomial algorithm is almost certainly impossible. Too bad; we have to solve the problem anyway.

The trivial solution is to try every possible assignment. We'll evaluate the running time of our 3SAT algorithms in terms of two variables: n is the number of variables in the input formula, and m is the number of clauses. There are 2^n possible assignments, and we can evaluate each assignment in $O(m)$ time, so the overall running time is $O(2^n m)$. Assuming no clause appears in the input formula more than once—a condition that we can easily enforce in polynomial time—then $m = O(n^3)$, so the running time is at most $O(2^n n^3)$.

Polynomial factors like n^3 are essentially noise when the overall running time is exponential, so from now on I'll write $\text{poly}(n)$ to represent any polynomial function of n ; in other words, $\text{poly}(n) := n^{O(1)}$. For example, the trivial algorithm for 3SAT runs in time $O(2^n \text{poly}(n))$.

One Clause

Recall that all 3CNF formulas have the following recursive structure

A 3CNF formula is either nothing
or (a clause with three literals) $\wedge$ (a 3CNF formula)

Suppose we want to decide whether some 3CNF formula Φ with n variables is satisfiable. Of course this is trivial if Φ is the empty formula, so suppose

$$\Phi = (x \vee y \vee z) \wedge \Phi'$$

for some literals x, y, z and some 3CNF formula Φ' . By distributing the $\wedge$ across the $\forall$ s, we can rewrite Φ as follows:

$$\Phi = (x \wedge \Phi') \vee (y \wedge \Phi') \vee (z \wedge \Phi')$$

For any boolean formula Ψ and any literal x , let $\Psi|x$ (pronounced “sigh given eks”) denote the simpler boolean formula obtained by setting x to TRUE and simplifying as much as possible. If Ψ has n variables, then $\Psi|x$ has at most $n - 1$ variables.

It’s not hard to prove by induction (hint, hint) that $x \wedge \Psi = x \wedge (\Psi|x)$, and therefore

$$\Phi = (x \wedge (\Phi'|x)) \vee (y \wedge (\Phi'|y)) \vee (z \wedge (\Phi'|z)).$$

Thus, for any satisfying assignment for Φ , either

- x is true and $\Phi'|x$ is satisfiable, or
- y is true and $\Phi'|y$ is satisfiable, or
- z is true and $\Phi'|z$ is satisfiable.

Each of the smaller formulas $\Phi'|x$, $\Phi'|y$, and $\Phi'|z$ has at most $n - 1$ variables. If we recursively check all three possibilities, we get the running time recurrence

$$T(n) \leq 3T(n-1) + \text{poly}(n),$$

whose solution is $O(3^n \text{poly}(n))$. So we’ve actually done *worse*!

But these three recursive cases are not mutually exclusive! If $\Phi'|x$ is *not* satisfiable, then x *must* be false in any satisfying assignment for Φ . So instead of recursively checking $\Phi'|y$ in the second step, we can check the even simpler formula $\Phi'|\bar{x}y = (\Phi'|\bar{x})|y$. Similarly, if $\Phi'|\bar{x}y$ is not satisfiable, then we know that y must be false in any satisfying assignment, so we can recursively check $\Phi'|\bar{x}\bar{y}z$ in the third step.

```

3SAT(Φ):
  if Φ = ∅
    return TRUE
  (x ∨ y ∨ z) ∧ Φ' ← Φ
  if 3SAT(Φ'|x)
    return TRUE
  if 3SAT(Φ'|\bar{x}y)
    return TRUE
  return 3SAT(Φ'|\bar{x}\bar{y}z)

```

The running time of this algorithm obeys the recurrence

$$T(n) = T(n-1) + T(n-2) + T(n-3) + \text{poly}(n),$$

where $\text{poly}(n)$ denotes the polynomial time required to simplify boolean formulas, handle control flow, move stuff into and out of the recursion stack, and so on.

How do we solve this recurrence? It isn’t simple enough to let us guess an accurate solution and check it by induction. Fortunately, there is a standard mechanical solution for all backtracking recurrences of the form

$$T(n) \leq \text{poly}(n) + \sum_{i=1}^k T(n - a_i),$$

where $k, a_1, a_2, \dots, a_k$ are constants. Assume without loss of generality that the constants a_i are indexed in non-increasing order: $a_1 \geq a_2 \geq \dots \geq a_k$. The fundamental theorem of algebra implies that the **characteristic polynomial**

$$r^{a_k} - \sum_{i=1}^k r^{a_k - a_i}$$

(where r is a formal variable) has exactly k complex roots, some of which may be equal. Let λ be the root with largest magnitude. Then the solution to the recurrence is $T(n) = O(\lambda^n \text{poly}(n))$. This technique is described in more detail in the appendix; in particular, we can actually derive *exact closed form* solutions to backtracking-style recurrences from the complete sequence of characteristic roots and base cases, if the polynomial terms and base cases are specified exactly.

The recurrence $T(n) = T(n-1) + T(n-2) + T(n-3) + \text{poly}(n)$ has characteristic polynomial $r^3 - r^2 - r - 1$, whose roots are approximately

$$1.83928676, \quad -0.419643337 + 0.60629073i, \quad -0.419643337 - 0.60629073i.$$

(Thank you, Wolfram Alpha!) The first root has larger magnitude than the other two, so we get the solution

$$T(n) = O(\lambda^n \text{poly}(n)) = O(1.83929^n).$$

(Notice that we cleverly eliminated the polynomial noise by ever so slightly increasing the base of the exponent.)

Pure Literals

We can improve this algorithm further by eliminating *pure* literals from the formula before recursing. A literal x is **pure** in if it appears in the formula Φ but its negation $\bar{x}$ does not. It’s not hard to prove (hint, hint) that if Φ has a satisfying assignment, then it has a satisfying assignment where every pure literal is true. If $\Phi = (x \vee y \vee z) \wedge \Phi'$ has no pure literals, then some clause in Φ contains the literal $\bar{x}$, so we can write

$$\Phi = (x \vee y \vee z) \wedge (\bar{x} \vee u \vee v) \wedge \Phi'$$

for some literals u and v (each of which might be y , $\bar{y}$, z , or $\bar{z}$). It follows that the first recursive formula $\Phi|x$ contains the clause $(u \vee v)$. We can recursively eliminate the variables u and v just as we eliminated the variables y and x in the second and third cases of our previous algorithm:

$$\Phi|x = (u \vee v) \wedge \Phi'|x = (u \wedge \Phi'|xu) \vee (v \wedge \Phi'|x\bar{u}v).$$

Our new faster algorithm is shown in Figure ???. The running time $T(n)$ of this algorithm satisfies the recurrence

$$T(n) = \text{poly}(n) + \max \left\{ \begin{array}{l} T(n-1) \\ 2T(n-2) + 2T(n-3) \end{array} \right\}$$

```

FASTER3SAT( $\Phi$ ):
  if  $\Phi = \emptyset$ 
    return TRUE
  if  $\Phi$  has a pure literal  $x$ 
    return FASTER3SAT( $\Phi|x$ )

   $(x \vee y \vee z) \wedge (\bar{x} \vee u \vee v) \wedge \Phi' \leftarrow \Phi$ 
  if FASTER3SAT( $\Phi|xu$ )
    return TRUE
  if FASTER3SAT( $\Phi|x\bar{u}v$ )
    return TRUE
  if FASTER3SAT( $\Phi|\bar{x}y$ )
    return TRUE
  return FASTER3SAT( $\Phi|\bar{x}\bar{y}z$ )

```

Figure B.1. A faster backtracking algorithm for 3SAT.

To solve this recurrence, we solve each case separately and take the larger of the two solutions. Here, the comparison is easy; the first case yields a *polynomial* bound, while the second case clearly yields an exponential bound, so for purposes of worst-case analysis, we can simplify the recurrence to $T(n) = O(\text{poly}(n)) + 2T(n-2) + 2T(n-3)$. The solution to this simpler recurrence is

$$T(n) = O(\lambda^n \text{poly}(n)) = O(1.76930^n),$$

where $\lambda \approx 1.76929235$ is the largest root of the characteristic polynomial $r^3 - 2r - 2$.

Local Search

We can get an even faster algorithm using a different backtracking strategy, called *local search*. Instead of constructing a satisfying assignment recursively, we search for a path through the set of all assignments that ends at a satisfying assignment.

The *Hamming distance* between two different assignments is the number of variables where the two assignments differ.² Suppose we could somehow magically find an assignment that is *almost* satisfying, meaning that its Hamming distance from some satisfying assignment is small. Then instead of searching over the entire space of 2^n assignments, we only need to look at the assignments that are close to the assignment we already have. Moreover, we can guide the search by considering only the clauses that are *not* satisfied by our current assignment.

To make this concrete, let A be our current assignment, and suppose there is a satisfying assignment A^* that has Hamming distance r from A . Consider any clause of Φ

²Richard Hamming was an early giant of computer and information science, who earned PhD in mathematics at Illinois in 1942, programmed IBM calculating machines for the Manhattan project in 1945, and then went on to create the first error-correcting codes. Arguably the greatest recognition of his achievements is that his name is attached to the number of indices where two bit vectors differ.

that is *not* satisfied by A ; at least one the three variables in that clause has a different value in A^* . Thus, if we change that particular variable's value, the resulting assignment has Hamming distance $r-1$ from A^* . Of course, we don't know which variable to change, so we have to try all three of them. And we don't know the target assignment A^* , so we just check at every stage of recursion whether our current assignment is satisfying. The resulting backtracking algorithm, shown in Figure ??, runs in $O(3^r \text{poly}(n))$ time.

```

LOCAL3SAT( $\Phi, A, r$ ):
  if  $A$  satisfies  $\Phi$ 
    return TRUE
  if  $r = 0$ 
    return FALSE

   $C \leftarrow$  any clause of  $\Phi$  that  $A$  does not satisfy
   $x, y, z \leftarrow$  variables of  $C$ 

   $A[x] \leftarrow \neg A[x]$ 
  if LOCAL3SAT( $\Phi, A, r-1$ ) = TRUE
    return TRUE

   $A[x] \leftarrow \neg A[x]$ 
   $A[y] \leftarrow \neg A[y]$ 
  if LOCAL3SAT( $\Phi, A, r-1$ ) = TRUE
    return TRUE

   $A[y] \leftarrow \neg A[y]$ 
   $A[z] \leftarrow \neg A[z]$ 
  return LOCAL3SAT( $\Phi, A, r-1$ )

```

Figure B.2. A local search algorithm for 3SAT.

It must be emphasized that the local search algorithm is *not* explicitly considering every assignment within Hamming distance r of the starting assignment A .

Now we apply the following trivial observation: In any satisfying truth assignment, either at least half the variables are TRUE or at least half the variables are FALSE. Thus, every satisfying assignment has Hamming distance at most $n/2$ from either the all-TRUE assignment or the all-FALSE assignment. So using the local search strategy, we can solve 3SAT in $O(3^{n/2} \text{poly}(n)) = O(1.73206^n)$ time.

♥ Random Starting Assignments

We can improve the local search algorithm further, at the expense of a small probability of a false negative, by repeatedly choosing *random* starting assignments and searching within a small radius of each.

First let's do some preliminary math. Let $V(r)$ denote the number of assignments within Hamming distance r of any fixed assignment. Suppose we run the local search algorithm starting with a random assignment; let $P(r)$ denote the probability that the algorithm finds a satisfying assignment (assuming one exists). A simple counting

argument implies that $P(r) \geq V(r)/2^n$. If we choose $N(r)$ random assignments and run the local search algorithm starting at each one, the probability of *not* finding a satisfying assignment (if one exists) is *at most* $(1 - P(r))^{N(r)}$. Thus, if we set $N(r) = n2^n/V(r)$, this failure probability is at most

$$(1 - P(r))^{n2^n/V(r)} \leq \left(\left(1 - \frac{V(r)}{2^n} \right)^{2^n/V(r)} \right)^n \leq e^{-n}.$$

The overall running time of our algorithm is

$$O(N(r) 3^r \text{poly}(n)) = O\left(\frac{n}{P(r)} \cdot 3^r \text{poly}(n)\right) = O\left(\frac{2^n 3^r}{V(r)} \text{poly}(n)\right).$$

To complete the analysis, we need to estimate the volume $V(r)$ and choose an appropriate value of r . For any constant $0 < \alpha < 1$, Stirling's approximation $n! \sim \sqrt{2\pi n} (n/e)^n$ implies the lower bound

$$V(an) = \sum_{i=0}^r \binom{n}{i} = \Omega(H(\alpha)^n / \sqrt{n})$$

where $H(\alpha)$ is the *binary entropy* function:

$$H(\alpha) = \left(\frac{1}{\alpha}\right)^\alpha \left(\frac{1}{1-\alpha}\right)^{1-\alpha}.$$

Thus, if we set $r = an$, the running time of our algorithm becomes

$$O\left(\frac{2^n 3^{an}}{H(\alpha)^n} \text{poly}(n)\right) = O\left((2(3\alpha)^\alpha (1-\alpha)^{1-\alpha})^n \text{poly}(n)\right)$$

We can find the value of α that minimizes the base of the exponential part of this upper bound using a bit of calculus.

$$\begin{aligned} \frac{d}{d\alpha} \ln((3\alpha)^\alpha (1-\alpha)^{1-\alpha}) &= \frac{d}{d\alpha} (\alpha \ln(3\alpha) + (1-\alpha) \ln(1-\alpha)) \\ &= \ln(3\alpha) - \ln(1-\alpha) = 0 \\ \Rightarrow \alpha &= 1/4. \end{aligned}$$

Finally, if we set $\alpha = 1/4$ (or equivalently, $r = n/4$), the algorithm runs in

$$O\left((2(3/4)^{1/4} (3/4)^{3/4})^n \text{poly}(n)\right) = O\left((3/2)^n \text{poly}(n)\right) \text{ time.}$$

Further Extensions

Naturally, these approaches can be extended much further. Since 1998, at least fifteen different 3SAT algorithms have been published, each improving the running time by a small amount; while a few of these improvements use pure backtracking, most of them are based on some variant of local search. For example, in 1999, Uwe Schöningh the running time of the randomized local search algorithm to $O((4/3)^n \text{poly}(n))$, still with an exponentially small error probability, by replacing the backtracking search with a random walk of length $3n$. As of 2016, the fastest deterministic algorithm for 3SAT runs in $O(1.3303^n)$ time³, and the fastest *randomized* algorithm runs in $O(1.30704^n)$ expected time⁴, but there is good reason to believe that these are *not* the best possible.

There is a thriving industry of practical SAT solvers, which combine backtracking, local search, random restarts, and other more sophisticated algorithmic techniques with domain-specific heuristics. With these tools in hand, most real-world instances of SAT are actually easy to solve in practice; modern SAT solvers routinely resolve instances with millions of variables and clauses. SAT solvers have become an indispensable tool in several areas of computing, including hardware and software verification, operations research, and computational biology. The practicality of SAT does not contradict the fact that the fastest algorithms require exponential time *in the worst case*; it just means that *the worst case* is rare in practice. Unfortunately, a proper discussion of practical techniques for SAT is beyond both the scope of this text and the expertise of its author.

B.2 Maximum Independent Set

Now suppose we are given an undirected graph G and are asked to find the size of the *largest independent set*, that is, the largest subset of the vertices of G with no edges between them. Once again, we have an obvious recursive algorithm: Try every subset of nodes, and return the largest subset with no edges. Expressed recursively, we obtain the following algorithm, where $N(v)$ denotes the *neighborhood* of v : the set containing v and all of its neighbors.

```

MAXIMUMINDSETSIZE( $G$ ):
  if  $G = \emptyset$ 
    return 0
  else
     $v \leftarrow$  any node in  $G$ 
     $withv \leftarrow 1 + \text{MAXIMUMINDSETSIZE}(G \setminus N(v))$ 
     $withoutv \leftarrow \text{MAXIMUMINDSETSIZE}(G \setminus \{v\})$ 
    return  $\max\{withv, withoutv\}$ .

```

³Kazuhisa Makino, Suguru Tamaki, Masaki Yamamoto. Derandomizing the HSSW algorithm for 3-SAT. *Algorithmica* 67(2):112–124 (2013)

⁴Timon Hertli. 3-SAT faster and simpler: Unique-SAT Bounds for PPSZ gold in general. *SIAM J. Comput.* 43(2):718–729 (2014).

Our algorithm exploits the fact that if an independent set contains v , then by definition it contains none of v 's neighbors. In the worst case, v has no neighbors, so $G \setminus \{v\} = G \setminus N(v)$. Thus, the running time of this algorithm satisfies the recurrence $T(n) = 2T(n-1) + \text{poly}(n) = O(2^n \text{poly}(n))$. Surprise, surprise.

This algorithm is mirroring a crude recursive upper bound for the number of *maximal* independent sets in a graph; an independent set is maximal if every vertex in G is either already in the set or a neighbor of a vertex in the set. If the graph is non-empty, then every maximal independent set either includes or excludes each vertex. Thus, the number of maximal independent sets satisfies the recurrence $M(n) \leq 2M(n-1)$, with base case $M(1) = 1$. We can easily guess and confirm the solution $M(n) \leq 2^n - 1$. The only subset that we aren't counting with this upper bound is the empty set!

Smarter Case Analysis

We can speed up our algorithm by making several careful modifications to avoid the worst case of the running-time recurrence.

Degree 0. If any vertex v has no neighbors, then $N(v) = \{v\}$, and both recursive calls consider a graph with $n-1$ nodes. But in this case, v is in *every* maximal independent set, so one of the recursive calls is redundant. On the other hand, if v has at least one neighbor, then $G \setminus N(v)$ has at most $n-2$ nodes. So now we have the following recurrence.

$$T(n) \leq \text{poly}(n) + \max \left\{ \begin{array}{c} T(n-1) \\ T(n-1) + T(n-2) \end{array} \right\} \\ = O(1.61804^n)$$

As before, the upper bound is derived by solving each case separately using characteristic polynomials and taking the larger of the two solutions. The first case gives us $T(n) = \text{poly}(n)$; the second case yields our old friends the Fibonacci numbers. Here and in later multi-case recurrences, I've typeset the worst case of the recurrence in **bold**.

Degree 1. We can improve this bound even more by examining the new worst case: some vertex v has exactly one neighbor w . In this case, either v or w appears in every maximal independent set. However, given any independent set that includes w , removing w and adding v creates another independent set of the same size. It follows that *some maximum independent set includes v* , so we don't need to search the graph $G \setminus \{v\}$, and the $G \setminus N(v)$ has at most $n-2$ nodes. On the other hand, if the degree of v is at least 2, then $G \setminus N(v)$ has at most $n-3$ nodes.

$$T(n) \leq \text{poly}(n) + \max \left\{ \begin{array}{c} T(n-1) \\ T(n-2) \\ \mathbf{T(n-1) + T(n-3)} \end{array} \right\} \\ = O(1.46558^n)$$

The base of the exponent is the largest root of the characteristic polynomial $r^3 - r^2 - 1$.

Degree ≥ 3 . Now the worst-case is a graph where every node has degree at least 2; we split this worst case into two subcases. If G has a node v with degree 3 or more, then $G \setminus N(v)$ has at most $n-4$ nodes. Otherwise (since we have already considered nodes of degree 0 and 1), *every* node in G has degree 2. Let u, v, w be a path of three nodes in G (possibly with u adjacent to w). In any maximal independent set, either v is present and u, w are absent, or u is present and its two neighbors are absent, or w is present and its two neighbors are absent. In all three cases, we recursively count maximal independent sets in a graph with $n-3$ nodes.

$$T(n) \leq \text{poly}(n) + \max \left\{ \begin{array}{c} T(n-1) \\ T(n-2) \\ T(n-1) + T(n-4) \\ \mathbf{3T(n-3)} \end{array} \right\} \\ = O(3^{n/3} \text{poly}(n)) \\ = O(1.44225^n)$$

The base of the exponent is $\sqrt[3]{3}$, the largest root of the characteristic polynomial $r^3 - 3$. The third case would give us a bound of $O(1.38028^n)$, where the base is the largest root of the characteristic polynomial $r^4 - r^3 - 1$.

Degree 2. Now the worst case for our algorithm is a graph with an extraordinarily special structure: *Every node has degree 2*. In other words, every component of G is a cycle. It is easy to prove that the largest independent set in a cycle of length k has size $\lfloor k/2 \rfloor$. So we can handle this case directly in polynomial time, with no recursion at all!

$$T(n) \leq \text{poly}(n) + \max \left\{ \begin{array}{c} T(n-1) \\ T(n-2) \\ \mathbf{T(n-1) + T(n-4)} \end{array} \right\} \\ = O(1.38028^n)$$

Again, the base of the exponential running time is the largest root of the characteristic polynomial $r^4 - r^3 - 1$. The final algorithm after all these improvements is shown in Figure ??.

```

MAXIMUMINDEPENDENTSETSIZE( $G$ ):
  if  $G = \emptyset$ 
    return 0
  else if  $G$  has a node  $v$  with degree 0 or 1
    return  $1 + \text{MAXIMUMINDEPENDENTSETSIZE}(G \setminus N(v))$      $\ll n-1$ 
  else if  $G$  has a node  $v$  with degree greater than 2
     $withv \leftarrow 1 + \text{MAXIMUMINDEPENDENTSETSIZE}(G \setminus N(v))$      $\ll n-4$ 
     $withoutv \leftarrow \text{MAXIMUMINDEPENDENTSETSIZE}(G \setminus \{v\})$      $\ll n-1$ 
    return  $\max\{withv, withoutv\}$ 
  else  $\ll \text{every node in } G \text{ has degree } 2 \gg$ 
     $total \leftarrow 0$ 
    for each component of  $G$ 
       $k \leftarrow$  number of vertices in the component
       $total \leftarrow total + \lfloor k/2 \rfloor$ 
    return  $total$ 

```

Figure B.3. A fast backtracking algorithm for MAXINDEPENDENTSET

Further Improvements

As with 3SAT, there are faster but increasingly complex algorithms for MAXINDEPENDENTSET. For example, in 1977, Bob Tarjan and Anthony Trojanowski published a complex backtracking algorithm that considers several dozen cases, each with its own running time recurrence. Most of these cases are clearly dominated by others; for example, even without solving the recurrences, it's easy to see that the recurrence $T(n) \leq \text{poly}(n) + T(n-4) + T(n-5)$ has a smaller solution than $T(n) \leq \text{poly}(n) + T(n-3) + T(n-5)$. After discarding all such dominated cases, Tarjan and Trojanowski are left with the rather impressive recurrence shown in Figure ???. The solution to this recurrence is $T(n) = O(1.25603^2) = O(2^{n/3})$, where the base of the exponent is the largest root of the characteristic polynomial $r^{10} - r^8 - r^2 - 2$ of the third case (in bold).

As of 2016, the fastest published algorithm for computing maximum independent sets runs in $O(1.2114^n)$ time⁵. However, in an unpublished technical report, Mike Robson describes a *computer-generated* algorithm that runs in $O(2^{n/4} \text{poly}(n)) = O(1.1889^n)$ time; just the description of this algorithm requires more than 15 pages.⁶

B.3 Dynamic Programming

Warmup: Bellman-Held-Karp TSP in $O(2^n \text{poly}(n))$ time. (Leave as exercise?)

⁵Nicolas Bourgeois, Bruno Escoffier, Vangelis Paschos, and Johan M. M. can Rooij. Fast algorithms for MAX INDEPENDENT SET. *Algorithmica* 62(1):382–415, 2010.

⁶John Michael Robson. Finding a maximum independent set in time $O(2^{n/4})$. Technical report 1251-01, LaBRI, 2001. (<http://www.labri.fr/perso/robson/mis/techrep.ps>).

★★★★

★★★★

$$T(n) \leq \text{poly}(n) + \max \left\{ \begin{array}{l} T(n-2) + T(n-6) \\ T(n-2) + 2T(n-8) \\ \mathbf{T(n-2) + T(n-8) + 2T(n-10)} \\ T(n-3) + T(n-5) \\ T(n-4) + T(n-6) + T(n-8) \\ T(n-4) + 2T(n-7) \\ T(n-4) + T(n-8) + 3T(n-11) \\ T(n-4) + 2T(n-8) + T(n-10) \\ T(n-4) + T(n-9) + 2T(n-12) \\ T(n-4) + 4T(n-10) \\ T(n-5) + 4T(n-9) \\ T(n-5) + 6T(n-11) + 4T(n-14) + T(n-17) \end{array} \right\}$$

Figure B.4. The (simplified!) running time recurrence for Tarjan and Trojanowski's independent set algorithm.

3-coloring in $O(2^n \text{poly}(n))$ time. Let $\text{OPT}(X) = \text{TRUE}$ if there is a 3-coloring in which every vertex in X is red. Then

$$\text{OPT}(X) = \begin{cases} \text{FALSE} & \text{if } X \text{ is not an independent set} \\ \text{TRUE} & \text{if } G[V \setminus X] \text{ is bipartite} \\ \bigvee_{v \in V \setminus X} \text{OPT}(X \cup \{v\}) & \text{otherwise} \end{cases}$$

Lawler improved the running time to $O(3^{n/3} \text{poly}(n))$ by observing that we only need to care about $\text{OPT}(X)$ when X is a *maximal* independent set. There are $O(3^{n/3})$ maximal independent sets, which can be listed in $O(3^{n/3} \text{poly}(n))$ time.

Alternatively, 3-coloring in $O(2^n \text{poly}(n))$ time by backtracking through the vertices in some fixed whatever-first order. Each vertex must be colored differently from its parent (and possibly other vertices) and without loss of generality, the first two nodes are red and green, respectively. So we consider only 2^{n-2} possible colorings, but memoization doesn't help. (Can we do better by looking in max-adjacency order?)

★★★★

Lawler's algorithm for chromatic number in $O((1+3^{1/3})^n \text{poly}(n))$ time: Let $\text{OPT}(X)$ denote the chromatic number of the subgraph $G[X]$ induced by the vertex subset $X \subseteq V$. Then

$$\text{OPT}(X) = \begin{cases} 0 & \text{if } X = \emptyset \\ 1 + \min \{ \text{OPT}(X \setminus I) \mid \text{maximal ind. set } I \text{ in } G[X] \} & \text{otherwise} \end{cases}$$

There are $O(3^{k/3})$ maximal independent sets of size k , which can be listed in $O(3^{k/3} \text{poly}(n))$ time.

★★★★

Other techniques to include?

- Horowitz-Sahni partitioning to solve SUBSETSUM in $O(2^{n/2} \text{poly}(n))$ time? (See also: 3SUM)
- Decision via counting via Inclusion-Exclusion?

Exercises

- (a) Prove that any n -vertex graph has at most $3^{n/3}$ maximal independent sets.
[Hint: Modify the MAXIMUMINDSETSIZE algorithm so that it lists all maximal independent sets.]
- (b) Describe an n -vertex graph with exactly $3^{n/3}$ maximal independent sets, for every integer n that is a multiple of 3.

- (a) Describe an algorithm for k SAT whose running time satisfies the recurrence

$$T(n) = 2T(n-1) - T(n-k+1) + \text{poly}(n).$$

- (b) Describe an algorithm for k SAT whose running time satisfies the recurrence

$$T(n) = T(n-1) + 2T(n-2) - 2T(n-k+1) + \text{poly}(n).$$

[Hint: Modify the backtracking algorithms for 3SAT described in the text. No, your algorithm will not make negative recursive calls.]

- Describe an algorithm that solves 3SAT in $O(\phi^n \text{poly}(n))$ time, where $\phi = (1 + \sqrt{5})/2 \approx 1.618034$. [Hint: Prove that in each recursive call, either you have just eliminated a pure literal, or the formula has a clause with at most two literals. What recurrence leads to this running time?]
- Describe an algorithm to determine whether a graph is 4-colorable in $O(2^n \text{poly}(n))$ time. [Hint: Consider red and green together, but separately from blue and yellow.]
- (a) Prove the following upper bound for all constant $0 < \alpha < 1$:

$$\binom{n}{\alpha n} \leq \sum_{i=0}^{\alpha n} \binom{n}{i} < \left(\left(\frac{1}{\alpha} \right)^\alpha \left(\frac{1}{1-\alpha} \right)^{1-\alpha} \right)^n$$

[Hint: Expand the expression $(\alpha + (1-\alpha))^n = 1$ using the binomial theorem.]

- (b) Prove the following lower bound for all constant $0 < \alpha < 1$:

$$\binom{n}{\alpha n} \geq \left(\left(\frac{1}{\alpha} \right)^\alpha \left(\frac{1}{1-\alpha} \right)^{1-\alpha} \right)^n \cdot \Omega(1/\sqrt{n})$$

[Hint: Use Stirling's approximation: $\sqrt{2\pi n} (n/e)^n \leq n! \leq 2\sqrt{\pi n} (n/e)^n$.]

Need a few more exercises.

